DEBRECENI EGYETEM

INFORMATIKAI KAR

SZAKDOLGOZAT

Csomagküldő szolgálat folyamatait támogató rendszer fejlesztése

Témavezető:
Dr. Major Sándor Roland
adjunktus

Készítette:
Domina Márk Zsolt
programtervező informatikus

DEBRECEN
2025

1 Tartalomjegyzék

1 Tartalomjegyzék 2

2 Bevezetés 5

2.1 Modern csomagküldő rendszerek 5

2.2 Célkitűzés 6

3 Csomagküldő szolgálatok oldalainak vizsgálata 6

3.1 Vizsgált csomagküldő vállalatok 7

3.1.1 GLS 7

3.1.2 ExpressOne 8

3.1.3 UPS 8

3.1.4 DHL 8

3.1.5 Foxpost 8

3.2 Eseti csomagrendelés folyamata 8

3.2.1 Első benyomások 8

3.2.2 Adatok megadása 9

3.2.3 Egyéb tulajdonságok 9

3.3 Árak megtekintése 10

3.4 Többnyelvűség 10

3.5 Támogatás 11

3.6 Csomagkövetés 11

3.7 Összegzés 11

4 Tervezés 13

4.1 Rendszerről általánosan 13

4.2 Felhasználói felület 13

4.3 Adminisztrációs felület 14

5 Alkalmazott Technológiák 14
5.1 Argon2id 15
5.2 Bootstrap 5 15
5.3 Cascading Style Sheets 16
5.4 Go 16
5.5 Go kit 17
5.6 GORM 19
5.7 Hypertext Markup Languange 19
5.8 JavaScript Object Notation 20
5.9 JSON Web Token 21
5.10 Paraglide JS 21
5.11 PostgreSQL 22
5.12 Progresszív Webalkalmazás 22
5.13 Prometheus 23
5.14 Svelte 23
5.15 SvelteKit 24
5.16 TypeScript 25
5.17 Universally Unique Lexicographically Sortable Identifier 25
5.18 YAML 26
6 Implementáció 26
6.1 Felhasznált eszközök 26
6.2 Backend 27
6.2.1 Backend felépítése 27
6.2.2 Szolgáltatások felépítése 28
6.2.3 Végpontok felépítése 32
6.2.4 Transzportok felépítése 32

6.2.5 Tárolók felépítése ... 33
6.2.6 Árazás kiszámítása ... 35
6.2.7 Authentikáció és autorizáció ... 35
6.3 Frontend ... 37
7 Felhasználói útmutató ... 43
7.1 Beüzemelés ... 43
7.1.1 Szoftverek telepítése ... 43
7.1.2 Adatbázis beállítása ... 44
7.1.3 Backend beállítása ... 44
7.1.4 Frontend beállítása ... 45
7.1.5 Prometheus beállítása ... 45
7.2 Rendszer használata ... 46
7.2.1 Felhasználó felület ... 46
7.2.2 Adminisztrációs felület ... 51
8 Összefoglalás ... 55
9 Köszönetnyilvánítás ... 56
10 Felhasznált könyvtárak ... 57
11 Irodalomjegyzék ... 57
12 Függelék ... 60
12.1 HTTP API Végpontok listája ... 60
12.1.1 Felhasználó API ... 60
12.1.2 Admin API ... 60

2 Bevezetés

Napjainkban az internet az életünk szerves részévé vált, a magyarok 96,4 százaléka használja legalább napi egyszer az internetet. [1] Az internetet nagyon sokszínűen használjuk, gondoljunk csak az e-mailekre, telefon- és videóhívásokra, a közösségi médiára, azonnali üzenetküldő alkalmazásokra. Az e-kereskedelem is egy fontos iparággá vált, ma már a bevásárlást is el tudjuk végezni online. 2024-ben a magyarok 61,7 százaléka adott három hónapban online végzi el legalább egy vásárlását. [2]

Az online piacterek bővülésével jelentősen megnőtt a kereslet a csomagküldő szolgáltatások iránt is. Ezek a szolgáltatások nem csak a webáruházak rendeléseinek kiszállításában nyújtanak segítséget, hanem az online apróhirdetések lebonyolításában, nagy mennyiségű iratok, például könyvelési dokumentumok mozgatásában, de már a magán jellegű csomagok, például ajándékok eljuttatásában is. A megnövekedett igényt jól mutatja, hogy csak a Magyar Postán a futárküldemények száma 2015 és 2024 között több mint a duplájára növekedett. [3] A nagy mennyiségű csomagok adminisztrációja, nyomon követése hagyományos, papír alapú módszerekkel lehetetlen feladat, ezért szükségessé vált az erre a tevékenységre specializált rendszerek fejlesztése.

2.1 Modern csomagküldő rendszerek

Egy modern csomagküldő vállalatirányítási rendszernek képesnek kell lennie a teljes szállítási folyamat automatizálására, és monitorozására. Egy ilyen alkalmazásnak sokféle feladatot kell ellátnia, például a csomagkiszállítás fuvarszervezését, szállító járművek és sofőrök menedzselését, megrendelések felvételét, a számlázások és kifizetések kezelését.

A csomagküldő vállalatoknak a versenyképességük fenntartása érdekében korszerű informatikai szolgáltatásokat kell nyújtaniuk a megrendelőik felé, mára már elképzelhetetlen, hogy a megrendelő ne tudja online leadni a rendelését vagy lekérdezni milyen státuszban van kézbesítés. A legújabb felületeken a felhasználó kedvére tudja változtatni a kiszállítás időpontját, vagy akár a kiszállítási címet is. A rendszeres vagy szerződéses csomagfeladóknak lehetőségük van a csomagküldő szolgálat API-án keresztül kezelni a megrendeléseiket, így a szállítmányozást akár a saját vállalatirányítási rendszerükbe is integrálhatják, ami nagyban megkönnyítheti a nagyforgalmú vállalkozások munkáját. Az eseti felhasználók segítésében nagyon fontos, hogy a felhasználói felület intuitív és felhasználóbarát legyen, és minél több

platformról el lehessen érni, ezért a legtöbb csomagszolgálat telefonos applikációt vagy reszponzív webfelületet biztosít a megrendelőinek. A legújabb trend a csomagautomaták használata, amiből a címzettek a nap 24 órájában átvehetik a küldeményeiket. Ezek a szükségletek komoly kihívásokat jelenthetnek informatikai háttérről.
A rendszerek hibás működése vagy váratlan leállása teljes mértékben megszakíthatják az üzlet működését, hiszen a csomagszállítás nehézkessé vagy lehetetlenné válhat, ami a vállalatoknak hatalmas anyagi és presztízs veszteséget okozhatnak. Emiatt úgy gondolom, hogy érdemes lehet ezeket a rendszereket kutatási céllal is megvizsgálni.

2.2 Célkitűzés

A szakdolgozatom keretében egy olyan rendszert fogok elkészíteni egy fiktív csomagküldő szolgálat részére, ami lehetővé teszi csomagküldések megrendelését. Ehhez létező vállalatok webalkalmazásait fogom megvizsgálni, hogy megállapítsam, milyen fontos funkcióknak kell rendelkeznie egy ilyen szoftvernek. A vizsgálataim eredményei és tanulmányaim alapján fogom az alkalmazást megtervezni, és megvalósítani. Az elkészülő alkalmazás elsődlegesen az eseti csomagküldés megrendeléséhez fog megoldást nyújtani, de egyéb szolgáltatásokat is fog tartalmazni az életszerűség kedvéért.
A fiktív vállalatnak nem fogok készíteni ál marketinganyagokat, mert a szakdolgozatomban az adminisztrációs folyamatokra fogok összpontosítani. A felhasználói felület mellett egy adminisztrációs felületet is fogok alkotni, amin keresztül a felhasználók csomagrendeléseit lehet majd kezelni. A rendszerem rendelésfeladástól a kiszállításig fogja végig követni a csomagküldés folyamatát web- és mobilfelületeken keresztül. Nem célom egy teljes vállalatirányítási rendszer létrehozása, az alkalmazásom egy kommunikációs felületet fog biztosítani a végfelhasználók és a vállalat adminisztrátorai között.

3 Csomagküldő szolgálatok oldalainak vizsgálata

A csomagküldő szolgálatok oldalainak vizsgálatának célja, hogy olyan közös tulajdonságokat találjak a csomagküldő szolgálatok weboldalain, amikből le tudom vonni a következtetést, hogy mik azok a fontos funkciók, amiket érdemes az általam fejlesztett alkalmazásban is megvalósítani. Nem csak az oldalak funkcióit, hanem a webfelületek kinézetét is vizsgálni

fogom, ezen belül a különböző űrlapok felépítését és működését, menüpontok és aloldalak elhelyezkedését is.

A weboldalokon belül, ha a vállalat nyújt rendszeres vagy szerződéses csomagfeladási szolgáltatást is, azt nem fogom vizsgálni, a kutatómunkám csak az eseti megrendelésekre fog kiterjedni. Azokat a felületeket nem fogom vizsgálni, amelyek eléréséhez tényleges rendelést kellene leadnom a csomagküldő szolgálatok felé. A vizsgálat során nem fogok foglalkozni azzal, hogy egyes szolgáltatók milyen technológiákat használnak a weboldalaik működéséhez, csak a felhasználói élményt fogom elemezni.

Azokat a hasznos funkciókat is megpróbálom kiszűrni, ami kevés oldalon található meg, de érdemes lehet őket felhasználnom a saját alkalmazásomban. Ha valamilyen funkciót különböző szolgáltatók merőben máshogyan valósítanak meg, akkor megpróbálok következtetést levonni, melyik megoldásnak van-e előnye vagy hátránya, esetleg melyiket tartom jobbnak.

3.1 Vizsgált csomagküldő vállalatok

A vizsgálat elvégzéséhez öt csomagküldő vállalat weboldalát választottam ki: GLS (https://ecsomag.hu), ExpressOne (https://expressone.hu/ebox), UPS (https://ups.com), DHL (https://dhl.com) és Foxpost (https://foxpost.hu). Ezeket a cégeket a mindennapi tapasztalataim alapján választottam, ezen vállalatok futárautóit gyakran látni akár Debrecen utcáin is, és többször kaptam már csomagot is a felsorolt szolgáltatókon keresztül. A Magyar Postát azért nem vizsgálom, mert jelenleg nem nyújtanak teljesen digitális csomagfeladási szolgáltatást, online csak címirat kitöltésre és nyomtatásra van lehetőség.

3.1.1 GLS

A GLS Cégcsoport, teljes nevén General Logistics Systems B.V. Európában és Észak-Amerikában nyújt szállítási megoldásokat. Jelenleg 50 országban üzemel leány- és partnervállalatokon keresztül, 23000 főnél több alkalmazottat foglalkoztat, és évi 926 millió csomagot továbbítanak. [4] A magyar leányvállalata, a GLS Hungary 89 depóból és egy csomagirányító központból áll, ami Alsónémediben található.

3.1.2 ExpressOne

Az ExpressOne az Osztrák Posta tulajdonában álló vállalat. A nemzetközi csomagforgalmat az anyavállalat segítségével bonyolítja le. A teljes Osztrák Posta csoport körülbelül 18 ezer alkalmazottal rendelkezik, és évi 269 millió küldeményt bonyolít le. [5]

3.1.3 UPS

A United Parcel Service, azaz a UPS egy nagymúltú csomagszállító szolgálat. 1907-ben alakult a cég az amerikai Seattle-ben, több mint százéves működésük során egy globális vállalattá nőtte ki magát. Ma több mint kettőszáz országban működik, világszerte 490 ezer embert foglalkoztatnak, és naponta 22,4 millió csomagot szállítanak ki. [6]

3.1.4 DHL

A DHL egy nemzetközi logisztikai vállalat, amely a nevét az alapítóinak vezetéknevei után kapta (Dalsey, Hillblom és Lynn). Habár az Egyesült Államokban alapították, a cég részvényeit a Német Posta tulajdonolja, a központjuk is Bonn-ban található. 84,2 milliárd eurós árbevételével Európa legnagyobb csomagküldő vállalata, körülbelül 600 ezer alkalmazottat foglalkoztatnak világszerte. [7]

3.1.5 Foxpost

A Foxpost egy Magyarországon alapított vállalat, amely csak belföldi csomagszállítással foglalkozik. Fő profiljuk a csomagautomatákon keresztüli csomagfelvétel és -átvétel, de hagyományos házhozszállítással is foglalkoznak. A cégnek nagyon fontos a környezettudatos működés, a fenntarthatóságukért több díjat is szereztek. [8]

3.2 Eseti csomagrendelés folyamata

3.2.1 Első benyomások

A csomagküldő szolgálatok kezdőlapjaira érkezve a felhasználót rengeteg tartalom fogadja: marketinganyagok, felhívások és számtalan információ lakossági és üzleti ügyfelek részére is. Mindegyik weboldalon egy eseti csomagrendelés megkezdéséhez a felhasználónak át kell navigálnia a megfelelő felületre. Minden esetben nagyon hasonlít ennek a menete, általában a navigációs sáv egyik menüpontjára kell kattintani.

A Foxpost kivételével az összes szolgáltató csomagrendszere el van választva valamilyen formában a többi aloldaltól. Az oldal URL-jéből is kikövetkezhető, hogy most már egy külön rendszerben vagyunk, például a GLS esetén a csomagrendelő rendszer egy főoldaltól teljesen különböző domain-en működik. A felület felépítése is megváltozik, a fejlécben csak a megrendelés folyamatához lényeges menüpontok maradnak. A csomagküldő rendszer elválasztása a fő oldaltól megkönnyíti a felület kezelését, mert a felhasználó figyelmét nem tudja elterelni fölösleges információ.

3.2.2 Adatok megadása

A csomagszállítás megrendelését minden oldalon űrlapok használatával kell elvégezni. Az űrlapokon különböző adatokat lehet megadni, amelyek fajtáik alapján csoportosítva vannak, például feladó, címzett, csomag adatai, számlázási cím, kiegészítő szolgáltatások. A rendelési adatok begyűjtésének folyamata a felhasználóktól különböző szolgáltatóknál különbözik, de alapvetően két csoportra oszthatóak:

Az első esetben az adatok megadása egy egybefüggő, hosszú oldalon zajlik, ahol minden információt egy űrlapon kell megadni. Ilyen módszerrel működik a GLS, ExpressOne, és a Foxpost weboldala.

A második esetben pedig az információk megadása lépésről-lépésre, külön oldalakon zajlik: a DHL először bekéri a cél és származási országot, utána egy felületen bekéri a csomag adatait, majd felajánlja a szállítási lehetőségeket, választás után pedig egy másik oldalra irányít át, ahol meg kell adni a pontos adatokat.

Mindkét megoldásnak meg van az előnye: az egybefüggő űrlapokat egyszerűbb kitölteni és átlátni azoknak a felhasználóknak, akik gyakran használják a rendszert, míg a több lépésre tagolt, rövidebb űrlapok használata egy varázsló-szerű felületet alkot, ami inkább a rendszert ritkábban használó embereknek kedvez. Mind a kettő módszernek jellemzője, hogy az űrlapok beviteli mezőihez a lehető legkevesebb magyarázat van megadva, több információért a felhasználónak egy kérdőjel vagy több gombra nyomva egy felugró menüben vagy egy másik oldalon kell többet megtudniuk az adott problémáról.

3.2.3 Egyéb tulajdonságok

Az összes oldalon van lehetősége a felhasználóknak felhasználói fiókot létrehozniuk, de a GLS, UPS és DHL oldalán nem kötelező regisztrálni csomagküldés megrendeléséhez.

Bejelentkezésre mindenhol van lehetőség a fejlécben található gomb segítségével. A bejelentkezett felhasználók mindegyik szolgáltató oldalán meg tudják tekinteni a leadott rendeléseiket.
Egy rendelésben több csomag megadására egyedül a GLS oldalán van lehetőség, ahol az űrlapon van lehetőség az adatokkal kitöltött csomagok megkettőzésére vagy törlésére. Az ExpressOne oldalán történő rendelés során nem kell megadni különböző csomagok méretét, csak a darabszámukat és az összsúlyukat. Mindegyik szolgáltató oldalán rendelés adatainak megadása után a felhasználó kap egy árkalkulációt, amit egy megerősítő oldalon vagy oldalrészen tud elfogadni.

3.3 Árak megtekintése

A felhasználónak a megfelelő csomagküldő szolgálat kiválasztása során az egyik legfontosabb szempont a fuvar díja, ezért külön megvizsgáltam, hogy mennyire könnyedén lehet az árakat megtekinteni vagy előzetes árajánlatot kérni. A legegyszerűbb dolgom a GLS és a Foxpost oldalán volt, mivel az üzleti modelljük szerint az árazás során csak a csomagméretet veszik figyelembe, a GLS-nél nemzetközi szállítás esetén csak a feladó/címzett országát kell figyelni, ezért az árakat egy táblázatból ki lehetett olvasni. A Foxpost oldalán ettől függetlenül található egy kalkulátor, ami inkább arra hívja fel a figyelmet, hogy adott csomagautomata mekkora csomag befogadására képes.
A többi szolgáltató a feladó és címzett pontos címéből és a csomagok súlyából számítja az árait, ezért egy ár kalkulátor használata elkerülhetetlen. A GLS, Foxpost, és ExpressOne esetén az Árak vagy Árkalkulátor oldal közvetlen elérhető a weboldalak fejlécéből, míg a DHL és az UPS esetében a fejléc egyik lenyíló menüjéből lehet odanavigálni.

3.4 Többnyelvűség

Az összes általam vizsgált szolgáltató oldalait több nyelven is el lehetett érni. Az UPS és a DHL oldalai elérhetőek az összes ország nyelvén, amelyekben működnek, a többi szolgáltató az angol és magyar nyelvet támogatta. Mivel a GLS a magyar felhasználóknak saját weboldalt üzemeltet, a Foxpost és az ExpressOne csak Magyarországon üzemelnek, ezért elegendőnek találom, hogy csak két nyelven lehet ezeket használni.
Az összes vállalat esetében a weboldalak navigációs fejlécében lehet változtatni milyen nyelven jelenik meg az oldal. A Foxpost fejlében az adott nyelv nemzeti zászlójára kattintva, a többi

oldalon szöveges gombokkal lehet nyelvet választani. Az UPS oldalán a választott nyelv teljes neve, a többi oldalon az adott nyelv országának kétjegyű ISO kódja látható. Az UPS oldalán a csomagleadás közben eltűnik a fejlécből a nyelvváltási lehetőség.

3.5 Támogatás

A felhasználóknak a szolgáltatások igénybevétele vagy akár már a csomagrendelés során is merülhetnek fel kérdései, vagy segítségre is lehet szüksége, ezért fontos, hogy a weboldalon hozzáférjen az ügyfélszolgálat elérhetőségeihez. A gyakran ismételt kérdések és válaszok elhelyezése az oldalon szintén hasznos lehet. Az öt szolgáltató mindegyike biztosítja ezeket a forrásokat a weboldalain, amelyeket általában a fejlécben található Kapcsolat, GYIK vagy Támogatás című menüpontokból el lehet elérni. Az UPS oldalán a csomagrendelés során a navigációs sáv hiánya miatt nem lehet ezekhez hozzáférni.

3.6 Csomagkövetés

Az összes weboldal rendelkezik csomagkövetési szolgáltatással, amelyeket felhasználók regisztráció nélkül is elérhetnek. A csomagazonosító megadásával a felhasználók le tudják kérni a csomag jelenlegi állapotát, például felvették-e már a csomagot, kiszállítás alatt van-e már, vagy esetleg nem sikerült kiszállítani. Mindegyik szolgáltató esetében nem csak egy különálló oldalon érhető el a csomagkövetés, hanem már a kezdőlapon is található egy erre alkalmas űrlap, amely átirányít a csomagkövetés oldalára. A GLS esetében egy nem létező URL útvonal megadásakor az erre adott *404 Not Found* válasz felületén is megtalálható egy ilyen űrlap. Mivel csomagok feladása nélkül nincs lehetőségem érvényes nyomonkövetési azonosítót megadnom, ezért azt nem tudtam megvizsgálni, hogy pontosan milyen adatokat lehet megtudni a csomagkövetés segítségével, illetve hogyan néz ki az információkat megjelenítő oldal.

3.7 Összegzés

A vizsgálatom során az eseti csomagfeladás teljes menetét és felhasználói felületét sikerült megvizsgálnom többféle szolgáltató oldalán keresztül. Sikerült sok olyan tulajdonságot, funkciót találnom, amik fontosak ahhoz, hogy az elkészülő alkalmazásom releváns legyen a valós életben működő rendszerek között. Ezek között olyanok is vannak, amiket az összes vizsgált vállalat weboldala, vagy csak az egyik implementálja.

A legtöbb egyező tulajdonságot az oldalak kinézetében találtam. A csomagfeladó rendszer fejlécében majdnem minden esetben ugyanazokat a funkciókat lehet elérni: az árkalkulátort, a támogatást, a bejelentkezést, a nyomonkövetést, illetve az oldal nyelvének beállítását. Egy táblázatban összegyűjtöttem azokat a tulajdonságokat, amiket érdemes lehet a saját alkalmazásomban is implementálni, és hogy melyik szolgáltatók oldalai alkalmazzák:

Jelmagyarázat:✓- Igaz ✗- Hamis ✓?-Részben igaz

Tulajdonság	GLS	ExpressOne	UPS	DHL	Foxpost
Csomagkezelő felület elválasztva a főoldaltól	✓	✓	✓	✓	✗
Árak vagy Árkalkulátor külön aloldalon is megtalálható	✓	✓	✓	✓	✓
Csomagrendelést egybefüggő űrlapokon kell leadni	✓	✓	✓	✗	✗
GYIK vagy Támogatás oldal elérhető a csomagleadás oldaláról	✓	✓	✗	✓	✓
Bejelentkezéshez hivatkozás a fejlécben	✓	✓	✓	✓	✓
Rendelés leadáshoz szükséges regisztráció	✗	✓	✗	✗	✓
Egyszerre több csomagot is lehet rögzíteni a rendelés során	✓	✓?	✗	✗	✗
Angol és magyar nyelven is elérhető a weboldal	✓	✓	✓	✓	✓
Az oldal nyelve a fejlécben állítható	✓	✓	✓?	✓	✓
Csomagkövetés csomagazonosítóval bejelentkezés nélkül	✓	✓	✓	✓	✓

4 Tervezés

4.1 Rendszerről általánosan

A rendszer fő célja csomagküldés rendelések befogadása, feldolgozása és tárolása. A backend egy API-t fog szolgáltatni a frontendnek, amin keresztül fog folyni az összes kommunikáció köztük. A backendnek naplóznia kell, és metrikákat fog gyűjteni az API-n keresztül haladó forgalomról. A rendszerbe bejelentkezett felhasználók képesek lesznek bejelentkezve maradni. A frontendnek reszponzívnak, és mobileszközökön is jól használhatónak kell lennie. A backend-et és a frontend-et is úgy kell megvalósítani, hogy platformfüggetlenek, és skálázhatóak is legyenek igény szerint. A backend monolitikus architektúrával fog elkészülni, de olyan programozási mintákat kell alkalmazni, hogy könnyen bővíthető legyen új funkciókkal. Az adatok egy erre alkalmaz adatbáziskezelő segítségével lesznek eltárolva.

4.2 Felhasználói felület

A felhasználóknak kötelező lesz a csomagmegrendelés használatához bejelentkezniük a felhasználói fiókkukkal, amit ők tudnak majd regisztrálni. Regisztráció után a felhasználó kap egy visszaigazoló e-mailt, amivel aktiválni tudja a felhasználói fiókját, enélkül nem fog tudni majd bejelentkezni.

A csomagszállítás rendelő felületen lehetőség lesz tetszőleges számú csomag adatainak rögzítésére, és a felvitt csomagok törlésére és megkettőzésére. Az adatok felvitele után a rendszer meg fog jeleníteni a felhasználónak egy megerősítő felületet ugyanazon az oldalon, ahol a csomagszállításra is megkapja az árajánlatot. A felhasználó ki fogja tudni listázni a saját leadott megrendeléseit.

Lehetőség lesz egy árkalkulátor használatára is, ahol az árajánlat megszerzéséhez sokkal kevesebb információk kell majd megadni. Egy támogatás oldalon a felhasználóknak lehetőségük lesz az ügyfélszolgálat elérhetőségét és a gyakran ismételt kérdéseket elolvasni. Az árkalkulátor és támogatás oldal használatához nem lesz szükség bejelentkezésre.

A csomagkövetés funkció segítségével a felhasználók információkat szerezhetnek a csomagok állapotáról a csomag azonosítójának megadásával. A csomagkövetés funkciót bejelentkezés nélkül is el lehet érni.

A felhasználók a különböző oldalak között a fejlécben található gombokkal fognak tudni navigálni. A felhasználók a weboldalt angol és magyar nyelven is megtekinthetik, és fejlécben tudják majd a kívánt nyelvet kiválasztani.

4.3 Adminisztrációs felület

Az alkalmazáshoz egy adminisztrációs felület is fog készülni, ahol az adminisztrátorok külön fiókkal be tudnak jelentkezni. Az adminisztrátorok képesek lesznek az összes rendelés megtekintésére, módosítására, és keresni is tudnak majd rendelésazonosító alapján. A rendelések alapértelmezetten kapnak egy aktív státuszt, amivel az adminisztrátorok tudni fogják ellenőrizni, hogy melyik rendelésekkel kell foglalkozniuk. Ezt a státuszt tudják majd deaktiválni és újra aktiválni is. Különálló csomagokra is tudnak fogni keresni, illetve módosítani tudják azokat. A csomagokhoz egy másik felületen lehet majd azokat az állapotokat hozzáadni, amiket a felhasználók a nyomon követésnél tudnak lekérni. Ehhez lesznek biztosítva előre megadott állapotok, de lesz lehetőség egyedi állapotok megadására is. Lehetőségük lesz az árazás módosítására.
A különböző felületek között a navigációs sáv segítségével fognak tudni lépkedni. Több adminisztrátort is létre lehet majd hozni, de erre csak egy alapértelmezett adminisztrátori fiók lesz csak képes. Az adminisztrációs felület is angol és magyar nyelven lesz elérhető.

5 Alkalmazott Technológiák

Mielőtt neki fogtam volna a terveim megvalósításának, ki kellett választanom, hogy milyen Tech Stack-et használjak fel az alkalmazás elkészítéséhez: ezen belül is egy adatbáziskezelő-rendszert az adatok perzisztálásához, egy programozási nyelvet a backend megvalósításához, és egy frontend keretrendszert. Ezek kiválasztásához több szempontot vettem figyelembe: olyan technológiákat próbáltam keresni, amelyekkel az alapképzés alatti kurzusaim során nem találkoztam még, mások minél kevesebbszer használták fel a saját szakdolgozatuk elkészítéséhez és terveim megvalósításához is alkalmasak kell, hogy legyenek.
Így esett végül választásul adatbáziskezelő-rendszernek a PostgreSQL, a backend programozásához a Go programnyelv, a frontend elkészítéshez pedig Svelte/SvelteKit felhasználói felület keretrendszer. Ezek mellé végül a backend leprogramozásának leegyszerűsítésére hozzáadtam a kialakult Tech Stack-emhez a Go kit nevű eszközrendszert is.

Ebben a fejezetben bemutatom a Tech Stack részeinek, és az implementáció során felmerülő egyéb technológiák lényegét, és hogy a mire használtam fel őket az alkalmazás fejlesztésében.

5.1 Argon2id

Az Argon2id egy jelszó hasheléshez és proof of work alkalmazásokhoz felhasználható algoritmus. Az Argon2 algoritmus fő variánsa. amely egy olyan függvény, ami futtatása nem csak processzor időt, hanem nagy mennyiségű memóriát is igényel. Az Argon2id az Argon2 másik két alverziójának az Argon2d és Argon2i kombinációja. Az Argon2d variáns adatfüggő, míg az Argon2i adatfüggetlen memória eléréssel működik, ezek közös használata védelmet nyújt az oldalcsatornás támadások ellen és megnehezíti a brute-force támadások véghezvitelét is. [9]

Az Argon2id-t az backenden eltárolandó és ellenőrizendő jelszavak hashelésére fogom használni.

5.2 Bootstrap 5

A Bootstrap 5 egy olyan CSS és JavaScript könyvtár, amely egy kész eszköztárat nyújt reszponzív és dinamikus oldalak készítéséhez. A Bootstrap hozzáadása egy projekthez nagyon egyszerű, mert csak hozzá kell adni két hivatkozást az adott HTML dokumentumhoz. A Bootstrap definiál olyan osztály attribútumokat, melyekkel a weboldalunk kinézetét változtathatjuk anélkül, hogy saját CSS kódot írnánk. Például, ha azt szeretnénk, hogy egy szöveg középre legyen igazítva, a *text-align: center* CSS tulajdonság alkalmazása helyett megadhatjuk a *text-center* értéket osztályként. Ezzel jelentősen felgyorsul webdizájn elkészítésének a folyamata, és jelentősen csökken a boilerplate CSS kódok mennyisége is. Tartalmaz egy komponens könyvtárat, ami olyan gyakran használatos elemekhez biztosít stílusokat, mint például navigációs sávok, lenyíló menük, harmonikák, kitűzők, felugró ablakok vagy kártyák. A könyvtár rácsrendszere lehetőséget ad egy olyan oldalelrendezés létrehozására, amely elemei reszponzívan egymás alá-fölé-mellé rendezik magukat a képernyőn található szabad hely függvényében. A Bootstrap fejlesztői a projekt nyílt-forráskódúságának révén hozzáférést adnak a forráskódhoz is, így az összes stílus igény szerint testreszabható.

A saját projektemben a Bootstrap eszköztár nagy részét fogom használni, célom, hogy a weboldalak a már bevált komponensek használatával készüljön el. Az elkészülő alkalmazásom fontos, hogy reszponzív legyen, ezért a rácsrendszert is alkalmazni fogom.

5.3 Cascading Style Sheets

A Cascading Style Sheets, röviden CSS, egy jelelőnyelv, amellyel megjelenítendő HTML dokumentumok stílusát adhatjuk meg a böngészőmotor számára. Egy CSS dokumentum olyan szabályokat tartalmaz, amelyek megadják, hogy adott HTML címkére milyen stíluselemek legyenek alkalmazva. Ha szűkíteni szeretnénk, hogy a szabályok melyik címkékre legyenek alkalmazva, címkék attribútumait, például osztályait és azonosítóját is megadhatjuk. Sokféle lehetőségünk van egy weboldal kinézetének módosítására, például megváltoztathatjuk a betűtípust, méretet, színt, betűstílust, a weboldal hátterét, a tartalmak elhelyezkedését, de animációkat is létre tudunk hozni. A szabályokban felmerülő méreteket megadhatjuk abszolút mértékegységekben, például pixelekben, de lehetőségünk van relatív mértékeket is megadni, amelyek alkalmazkodnak az adott megjelenítő eszköz méreteihez. Médialekérdezések segítségével reszponzív oldalakat is készíthetünk, amelyekkel egyedi szabályokat adhatunk meg a megjelenítő eszköz típusára (kijelző, nyomtató stb.) és méreteire vonatkozóan. CSS kódot elhelyezhetünk külön stíluslapokon, a HTML dokumentumba ágyazva a <*style*> címke segítségével, vagy csak egy címkére alkalmazva egy *style* attribútumban is. Ha nem adunk meg egyedi stílust a HTML elemeinkre, akkor a böngésző a saját beépített stíluslapját használja fel. A CSS szabványok készítésével is a W3C foglalkozik.
Weboldalak dizájnját nem lehet CSS nélkül elkészíteni, ezért én is használni fogom, hogy egyedi kinézetet készítsek a webalkalmazásomhoz.

5.4 Go

A Go egy nyílt forráskódú, általános célú programozási nyelv, amelyet a Google fejlesztett ki. Legfőbb tulajdonsága, hogy nyelvi szinten támogatja a konkurens programozást, az úgynevezett goroutine-ok és csatornák segítségével. Egy compiler-es nyelv, szintaxisa a C nyelvéhez hasonlít, emellett erősen típusos is. A fordítóprogram a szintaktikai hibák mellett rákényszeríti a programozót, hogy jobb kódot írjon: például a nem használt változók deklarációi, és a fölöslegesen beimportált függőségek is fordító hibát okoznak. Különlegessége a nyelvnek, hogy garbage-collection-nel rendelkezik, de ettől még lehetőség van pointerek használatára is, ezáltal lehetőség adódik magas szintű programkódok alacsony szinten való optimalizálására. Egy modern programozási nyelvtől megszokott primitív típusok mellett a Go komplex számokat és az asszociatív tömböket is nyelvi szinten támogatja. A Go-val nagyon

egyszerű a függőségek kezelése, minden programot egy csomagként kell kezelni, és a Go csomagkezelője képes a csomagok automatikus importálására, verziókezelésére, illetve a tranzitív függőségek menedzselésére is. A pkg.go.dev weboldalon keresztül lehetőség van az összes publikált csomag dokumentációjának és forráskódjának elérésére.

A Go támogatja az objektum-orientált programozást is, de a hagyományos OOP nyelvektől eltérő módon van rá lehetőség. A nyelvben nincsenek osztályok, csak struktúrák és öröklődés sincs, objektumhierarchia kialakítása csak aggregációval és kompozícióval lehetséges. Egy struktúra mezőinek lehet tag-eket megadni, amelyek reflekción használatával kibővítik a struktúra képességeit. Interfészek implementálása implicit módon történik, egy struktúra akkor implementál egy interfészt, ha az összes, interfész által definiált metódust megvalósítja. Ezek a tulajdonságok rákényszerítik a programozókra a GoF alapelvek használatát: az interfészre való programozás, és a kompozíció öröklés helyetti használat a Go felépítése miatt elkerülhetetlenné válik. A Go a funkcionális programozás elemeit is hordozza: lehetőség van függvény típusok és anonim függvények megadására is. Egységtesztek készítésére is van lehetőség gyári eszközökkel. [10]

A Go programozási nyelvet a backend elkészítéséhez fogom használni. A nyelv sztenderd könyvtára rengeteg hasznos csomaggal rendelkezik, amelyek felhasználásával a klasszikus programozási problémákat, pl. HTTP szerver készítése, JSON-ök feldolgozása, nem kell újra megvalósítanom. Emellett a közösségi könyvtáraknak is nagy hasznát fogom venni, a nyelv népszerűségének köszönhetően nem nagyon található olyan technológia, amihez ne készült volna már Go implementáció is.

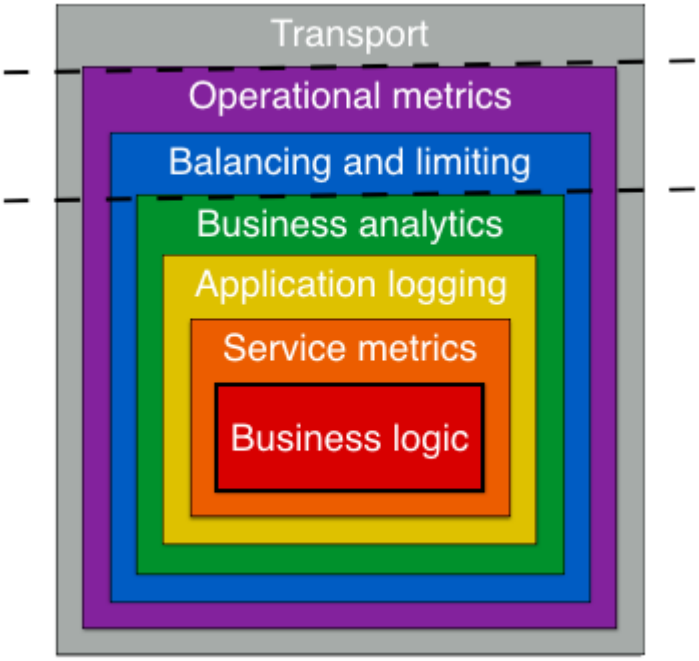
5.5 Go kit

A Go kit egy eszköztár, amellyel mikroszolgáltatás architektúrát lehet egyszerűen implementálni egy Go programnyelvű alkalmazásban. Fő célja, hogy mikroszolgáltatások készítésének során felmerülő funkcionalitásokhoz, például naplózáshoz, szolgáltatásfelfedezéshez, monitorozáshoz nyújtson egy programozási modellt, anélkül, hogy a fejlesztőket valamilyen keretrendszer vagy platform használatára kényszerítse. A Go kit felépítésében nem egy MVC keretrendszer, hanem három rétegből épül fel: szállítási, végpont és szolgáltatás rétegből. A kérések ebben a sorrendben haladnak végig a rendszeren, és a válaszok pont a fordított útvonalat járják be.

A szállítási réteg feladata, hogy egy konkrét hálózati elérést valósítson meg a végpontok számára, a kérések és válaszok kódolása is feladatai közé tartozik. Egy végponthoz több szállító is tartozhat. A Go kit transzport csomagja több kész implementációt tartalmaz, például a HTTP, gRPC használatához, de a fejlesztő is tud létrehozni saját transzportokat.

A végpont réteg a szolgáltatások vezérlőjeként működik. Egy végpontban élnek azok biztonsági logikák, amelyek lehetővé teszik a szolgáltatások zavartalan elérését, például terheléselosztás és korlátozás.

A szolgáltatási rétegben található az üzleti logika megvalósítása. Olyan egyéb feladatok ellátásért felelős, mint az üzleti analitikák készítése, naplózás, szolgáltatási metrikák begyűjtése. Go kit használata során a szétválasztás elvét kell követni, egy szolgáltatás különböző rétegei egy közös interfészt implementálnak, amelyeket a middleware (díszítő) programozási minta használatával lehet összekapcsolni egy egész szolgáltatássá.



1. ábra: Go kit architektúrája

A Go kit Inversion of Control minták használata helyett az egész alkalmazást a csomag main függvényében kell összekapcsolni. Előnye, hogy az összes függőség, szolgáltatások részei, végpontok és transzportok explicit vannak komponálva, ezáltal kevésbé támaszkodik a program globális állapotokra, így az egységek tesztelése és karbantartása is egyszerűbbek. Hátránya, hogy nagyobb alkalmazások esetén a main függvény nagyon nagyra nőhet.

Az adatok perzisztálásához tároló interfészek létrehozása javasolt, amelyeken belül egyedi implementációt tudunk adni az adatbázis hozzáférésekhez, és egyben a működésüket elválasztjuk az üzleti logikától is. [11]
A Go kitet a rendszerben a backend elkészítéséhez fogom használni. Habár elsődlegesen ezt az eszköztárat nem monolitikus alkalmazások fejlesztéséhez tervezték, a rétegelt architektúra és könnyen cserélhető komponensek használatával egy jól karbantartható és fejleszthető programot lehet írni, illetve a lehetőség is fennáll arra, hogy a mikroszolgáltatás architektúrára is át lehessen helyezni igény szerint.

5.6 GORM

A GORM egy Objektum-relációs leképzés könyvtár a Go programnyelvhez. Ez a könyvtár biztosít egy alapmodellt, amit egy Go *struct*-ban használva automatikusan hozzáad olyan funkciókat, mint a puha törlés, elsődleges kulcs kezelése, létrehozás vagy módosítás időpontjának tárolása. Alapértelmezetten minden SQL lekérdezést tranzakcióként futtat, így hiba esetén kisebb valószínűséggel sérülnek meg az adatok az adatbázisban. A GORM ORM létére automatikusan létrehozza a megadott objektum modellek alapján az adatbázis sémáját, és automatikus adatbázis-migrációt is támogat: a legnépszerűbb relációs adatbázisokhoz, köztük a PostgreSQL-hez is biztosít illesztőprogramokat, így fejlesztés és a modellezés során nem kell foglalkozni az adatbázis specifikus tényezőkkel.
A rendszerem fejlesztéséhez azért fogom használni a GORM-ot, mert megegyszerűsíti a fejlesztést: könnyedén megváltoztathatom a programkódomban a modellek felépítését anélkül, hogy DDL utasításokat kelljen kiadnom az adatbázisban.

5.7 Hypertext Markup Languange

A Hypertext Markup Languange vagy HTML egy jelelőnyelv, amivel a böngészőben megjeleníthető dokumentumokat lehet készíteni. Egy HTML dokumentum különböző címkékből épül fel, amelyekkel a különböző adatelemek megjelölhetőek a böngészőmotor számára, hogy milyen tartalomként értelmezze azokat. A címkék segítségével elhelyezhetünk a weboldalon fejléceket, bekezdéseket, felsorolásokat vagy táblázatokat, nem szöveges tartalmakat például képeket vagy videókat, de akár interaktív elemeket is, például gombokat, űrlapokat vagy hiperhivatkozásokat is.

A címkék elláthatóak különböző attribútumokkal, amelyekkel egyébként azonos típusú tagek is külön testreszabhatóak. Speciális attribútumok az osztályok és az azonosítók, a segítségükkel egyedi címkékre hivatkozhatunk külső stíluslapokban vagy szkriptekben. A megjelenítendő tartalmak mellett egy HTML dokumentum egyéb metaadatokat is tartalmazhat, amelyek nagy mértékben módosíthatják a weboldal működését, például hivatkozások kliens oldali szkriptekre, stíluslapokra, keresőmotorok számára hasznos kulcsszavak.
A HTML az XML jelelőnyelv egy sémájára épült, de modern HTML5 szabványt már nem lehet csak XML szabályokkal jellemezni. A HTML első verziói tartalmaztak olyan címkéket, amelyek a megjelenítendő tartalom stílusát is módosították, de a HTML5-ben már csak szemantikai jelentésük lehet. A technológia szabályozásával a W3C, azaz World Wide Web Consortium foglalkozik.
Webalkalmazások készítéséhez megkerülhetetlen eszköz, ezért az elkészülő alkalmazásomban is alkalmazni fogom a weboldalak elkészítéséhez.

5.8 JavaScript Object Notation

A JavaScript Object Notation, mindennapi nevén JSON egy szöveges formátum, amit főleg adatátvitelek során használunk. Egy JSON szöveg kulcs-érték párokból épül fel. A JavaScript szintaxisának egy részhalmaza, hatféle értéket tud ábrázolni: számokat, szövegeket, logikai értékeket, tömböket, objektumokat és nullértéket. A JSON-öknek sokféle felhasználásuk van: használjuk őket adatátvitelek, például API lekérdezések vagy válaszok során, konfigurációs fájlokként. A legtöbb modern programozási nyelvnek van beépített könyvtára a JSON szövegek feldolgozására, a dokumentum-orientált NoSQL adatbázisokban az egyik leggyakrabban használt dokumentum fajta, de már relációs adatbázisokban, például a PostgreSQL-ben is van lehetőség JSON típusok tárolására. A JSON használata XML helyett előnyösebb lehet, mert ugyanaz az adat JSON-ben ábrázolva kisebb, mint XML-ben, a JSON emberek számára könnyebben olvasható, és a programok is gyorsabban dolgozzák fel a JSON dokumentumokat, hátrányai, hogy a JSON nem támogatja a megjegyzések elhelyezését, az XML többféle adattípust támogat, például dátumokat és megkülönböztetett számformátumokat. A JSON és az XML esetén is van lehetőség sémák alapján a dokumentumok érvényességének ellenőrzésére, de míg az XML a saját formátumába beépítve támogatja ezt, addig a JSON esetén csak egy külső eszköz, a JSON Schema használatával lehet elérni.

Az alkalmazásomban a JSON-t a szerveroldali API adatátviteli formátumaként fogom használni, mert könnyű prototípusokat készíteni vele, esetemben nincsen szükség bináris formátumok használatára, és a Go programozási nyelv nagyon jó eszköztárat nyújt a JSON-ök parszolására és szerializálására.

5.9 JSON Web Token

A JSON Web Token, röviden JWT egy olyan technológia, amivel két fél között biztonságosan lehet állításokat megosztani. Egy JWT-nek vannak az IANA által regisztrált állítás fajtái, de egyedi mezőket is meg lehet adni, ahol viszont számolni kell az ütközésekkel. Hétféle regisztrált állítást lehet beállítani: az állítások alanyát, célközönségét, a JWT kiadóját, a kiadás időpontját, a JWT azonosítóját, illetve, hogy mikortól és meddig érvényesek az állítások. A JWT állításait egy JWE-be vagy egy JWS-be kell ágyazni, azaz digitálisan aláírni vagy titkosítani kell őket. [12]
Az alkalmazásomban a JWT-ket a felhasználók és az adminisztrátorok autentikálására és autorizálására fogom használni HTTP Bearer tokenekként. Használatuk előnye, hogy állapotmentesek, nem szükséges a kliens és a szerver között egy munkamenet fenntartására, hátránya, hogy egy JWT-t nem lehet utólag érvényteleníteni, ehhez az elutasítandó JWT-k azonosítóinak szerver oldali eltárolására van szükség.

5.10 Paraglide JS

A Paraglide JS egy i18n, azaz egy internacionalizációs JavaScript könyvtár, ami egy fordítóprogramot is használ a szövegek nyelvi fordításának elkészítéséhez. A fordító a megadott szövegekből olyan függvényeket generál, amelyek a meghívásukkor azon a nyelven adják vissza szöveget, amit a hívó környezetéből észlelnek. Ezek a függvények típusbiztosak is, így programkódban való használatuk is jobb. [13] A könyvtár lehetővé teszi lokalizált URL útvonalak és paraméteres szövegek megadását is. [14] Előnye, hogy adott weboldal megnyitásakor csak az arra az oldalra szükséges fordításokat küldi el.
A projektemben azért lesz hasznos ez a könyvtár, mert nem kell külön kódot írnom az angol-magyar nyelvű oldalak készítéséhez, és hivatalosan támogatja a SvelteKit-be való integrációját is.

5.11 PostgreSQL

A PostgreSQL egy nyílt forráskódú objektum-relációs adatbáziskezelő rendszer, amely a negyedik legnépszerűbb a világon a DB-Engines rangsorolása alapján. [15] A PostgreSQL számtalan olyan funkcióval rendelkezik, ami miatt érdemes használni. Csak pár fontosabb tulajdonság példának: a primitív adattípusok mellett strukturált adatok és dokumentumok tárolására is alkalmas, hagyományos indexelési módszerek mellett összetettebb megoldásokat is kínál, támogatja az egymásba ágyazott tranzakciók használatát, ACID elveknek megfelel. Lehetőség van kiegészítők programozására is többféle nyelven, amellyel a PostgreSQL képességei növelhetőek. A népszerű Post-GIS nevű bővítmény térinformatikai adatok tárolásának és feldolgozásának lehetőségével bővíti ki az adatbáziskezelő rendszert. [16] A PostgreSQL az összes népszerű platformra elérhető, és az évről-évre növekvő népszerűségével egy ideális választás, ha adatokat akarunk tartósan tárolni.

5.12 Progresszív Webalkalmazás

Egy progresszív webalkalmazás, mozaiknevén PWA egy olyan webalkalmazás, amely olyan szintű felhasználói élményt nyújt, mint egy specifikus platformra kifejlesztett. Ezt úgy éri el, hogy kombinálja a két világ legjobb tulajdonságait: kihasználja, hogy webalkalmazásként platformfüggetlen és egy kódbázisból működik, de úgynevezett service worker-ek segítségével offline állapotban és a háttérben is tud futni a kliens eszközén, telepíteni lehet és eszközöktől függően integrációra is képes a felhasználó operációs rendszerével, illetve más alkalmazásokkal. [17]

Ahhoz, hogy egy PWA telepíthető legyen, az oldal HTML dokumentumában be kell hivatkozni egy web app manifestet, ami egy jegyzék, amely különböző információkat tartalmaz a telepítendő alkalmazásról, mint például a neve, leírása, hálózati elérhetősége vagy az alkalmazáshoz tartozó ikonok helye. Lehetőség van arra is, hogy egy PWA-t natívan feltöltsük olyan népszerű alkalmazásboltba, mint a Google Play Store vagy a Microsoft Store. Az IOS App Store-ba egyelőre csak egy wrapper készítésével lehet feltölteni. [18]

Az rendszerem elkészítése során úgy döntöttem, hogy natív telefonos alkalmazások helyett PWA funkcionalitást fogok inkább megvalósítani, mivel az alkalmazás reszponzív oldalait így fel tudom használni, ezzel leegyszerűsítve és felgyorsítva a fejlesztést. A technológia használatának hátránya, hogy lassabbak a natív mobil alkalmazásoknál. Az alkalmazásom

aránylag kevés kliens oldali számítást igényel, ezért a PWA is megfelelő választás ebben az esetben.

5.13 Prometheus

A Prometheus egy Go programnyelven írt rendszerfelügyelő és figyelmeztető eszköztár. A Prometheus a különböző metrikákat idősoros adatokként gyűjti be és tárolja el, amik feldolgozásával megfigyelhetőek és ábrázolhatóak a hozzákapcsolt alkalmazások metrikái vagy hibajelzéseket is lehet automatikusan indítani. Az adatok begyűjtése a HTTP protokollon keresztül zajlik. A legfontosabb építőelemei: a fő Prometheus szerver, ami begyűjti az adatokat; kliens könyvtárak, amivel a mérőeszközök beépíthetőek különböző programnyelven írt alkalmazásokba; az *alertmanager*, ami felmerülő hibajelzéseket kezeli, és az exportálók, amivel az adatok kikérhetőek más szolgáltatások számára is a PromQL lekérdező nyelv segítségével. A Prometheus szerver biztosít egy webes felületet is az adatok kikérésére és vizualizálására is. [19]

Az én rendszeremben is szükség van a szolgáltatások monitorozására, és a Prometheus azért kiváló választás nekem, mert az metrikák begyűjtéséhez kliens könyvtárak a natúr Go-hoz és a Go kit-hez is találhatóak.

5.14 Svelte

A Svelte reaktív felhasználói felületek építésére alkalmazható keretrendszer, amely egy fordítóprogramot használ a fejlesztő által definiált HTML, CSS, és TypeScript kódokból álló komponensekből böngészőkre optimalizált JavaScript könyvtárak készítésére. [20]

Minden komponens egy .svelte kiterjesztésű fájlból áll. Egy komponensben három szekciót lehet megadni: a komponens TypeScript szkriptjét, a komponenshez tartozó HTML jelöléseket, és a komponens egyedi CSS stílusát. [21] A komponensek egymásba ágyazhatóak, így a programkód újrafelhasználhatósága is növelhető. A komponenseknek attribútumokat is tudunk megadni, melyet a Svelte propoknak hív, amikkel módosíthatjuk egy beágyazott komponens megjelenését vagy tartalmát.

A Svelte másik fontos képessége a sablonok használata. Sablonok segítségével egy komponens HTML kódjába reaktív elemeket tudunk elhelyezni ciklusok, elágazások, speciális kulcsszavak használatával, amelyeket szintén a fordítóprogram dolgoz fel és alakít HTML kóddá. Lehetőségünk van például tartalmak feltételes megjelenítésére, tömb elemeinek ciklikus

megjelenítésére, űrlapmezők értékeit tudjuk egy változó értékeihez kötni, egyedi animációkat is meg tudunk jeleníteni, de kapcsos zárójelek használatával tetszőleges TypeScript kódot is ki lehet értékelni.

A keretrendszer új verziójában, a Svelte 5-ben bevezettek egy új nyelvi elemet, a rúnákat, amelyekkel a komponenseken belülről is lehet vezérelni a Svelte fordítóprogramját. Példaként a *$state* rúnával reaktív állapotokat tudunk létrehozni, a *$props* rúnával pedig a komponenseknek megadott attribútumokat tudjuk elérni. [22]

A Svelte-t az alkalmazásom készítése során nagy segítséget fog nyújtani, mert felgyorsítja a fejlesztés folyamatát, könnyen lehet reaktív oldalakat kialakítani vele. A Svelte eszköztárának kiterjedtsége miatt nem kellett rengeteg boilerplate JavaScript kódot megírnom, hanem inkább az üzleti logika megvalósítására összpontosíthattam.

5.15 SvelteKit

A SvelteKit egy webalkalmazások fejlesztésére alkalmas keretrendszer, amely a felhasználó felület elkészítéséhez a Svelte-t használja. A SvelteKit a működéséhez nagy részben a SvelteKit project felépítésre hagyatkozik. Az útválasztása fájlrendszer alapú, amely a *routes* alatti könyvtárak alapján építi fel a HTTP útvonalakat. Adott útvonalhoz meg tudjuk adni a megjelenítendő Svelte komponenst, illetve egyedi szerver és kliens oldali szkripteket is. Layoutokat is létre lehet hozni, amelyek akkor lehetnek hasznosak, ha több weboldal használ közös elemeket, például navigációs sávokat. [23]

A szkriptek elkészítéséhez a SvelteKit biztosít olyan típusokat, amelyek implementációjával különböző funkciókat tudunk könnyen megvalósítani. A PageLoad használatával a szkripthez kapcsoló komponensbe tudunk tetszőleges adatokat betölteni, az Actions típussal pedig egyedi űrlapfeldolgozókat lehet készíteni. Vannak olyan funkciók, amelyek csak szerver, csak kliens vagy mindkettő típusú szkriptben is elérhetőek.

Az alkalmazásban történő specifikus események kezelésére kampókat lehet definiálni. A kampók képesek az útvonalak, fetch-ek HTTP kéréseinek elkapására, amelyeket módosítani is tud, így a kampókon belül lehetséges autorizációt igénylő weboldalak, átirányítások kezelése. Az elkapás során egyedi adatokat is hozzá tudunk adni a kérésekhez a *locals* mező kitöltésével, amelyhez a szerver oldali szkriptek az útvonal betöltése során felhasználhatnak. [24]

A SvelteKit a Vite webszervert használja a webalkalmazások buildeléséhez, és a fejlesztést is megkönnyíti például hot swap használatával. A projektek építéséhez rendelkezésre állnak

különböző adapterek, amelyek segítségével különböző platformokra tudjuk könnyedén beüzemelni az alkalmazásunkat. Van lehetőség statikus oldal, node serverekhez alkalmazható fájlok generálására, de különböző felhőszolgáltatásokhoz, például Cloudflare-hez, Vercel-hez vagy Netlify-hoz is tudja a webalkalmazást telepíteni. Egyedi adaptereket is le lehet programozni. [25]
Habár a SvelteKit képes teljes értékű backendként is működni, én a fejlesztés során úgy döntöttem, hogy csak a frontend elkészítéséhez fogom használni. Olyan funkciókat, mint az útvonalválasztás, űrlapadatok kezelése és adatok lekérése API-ról sokkal egyszerűbb SvelteKit-tel megvalósítani, mintha Go-ban írtam volna meg ezeket.

5.16 TypeScript

A TypeScript egy erősen típusos programozási nyelv, ami a JavaScript szintaxisán alapul. A TypeScript kiegészíti a JavaScriptet egy fordító-idejű típusellenőrzővel, az így elkészülő kód pedig típusbiztossá válik. A típusellenőrző kikövetkeztetheti egy adott objektum típusát a programkörnyezetből, de a fejlesztő típusannotációkat tud meg adni függvényparaméterekre, változókra, ezekkel a módszerekkel a fordító kényszeríti ki a típusbiztonságot. Lehetőség van saját típusok létrehozására is interfészekkel.
A projektem során a TypeScript használata nem megkerülhető, mert SvelteKit fejlesztése ezen a programozási nyelven támogatott, de nagy segítséget fog nyújtani általánosabb programozási problémák, például adatátviteli objektumok, röviden DTO-k használatához is.

5.17 Universally Unique Lexicographically Sortable Identifier

Az Universally Unique Lexicographically Sortable Identifier, röviden ULID egy olyan azonosító formátum, amit az UUID alternatívájának hoztak létre. Az ULID egy 26 karakter hosszú lánc, ami egy 128 bites bináris adat Crockford base32 algoritmusával kódolva jön létre. Az ULID binárisan két részből épül fel: az első 48-bit egy UNIX időbélyeget tárol, a maradék 80-bit pedig véletlenszerű értékeket tartalmaz. Különlegessége, hogy UUID-kal ellentétben karakterlánc reprezentációjukban lexikografikusan rendezhetőek, 10 karakterrel rövidebbek, kis/nagybetűre nem érzékenyek, nem tartalmaznak speciális karaktereket így URL-ekben is biztonságosan használhatóak.

A projektemben az ULID technológiát különböző azonosítók, például csomagokhoz és rendelésekhez fogom használni, mert így könnyen tudok rövid, fix hosszúságú, de biztosan egyedi kódokat generálni.

5.18 YAML

A YAML egy adatleíró nyelv, ami leggyakrabban konfigurációs fájlok formátumaként használatos. A neve *YAML Ain't Markup Languange* rekurzív mozaikszóból ered. A YAML ugyanúgy kulcs-érték párokat tárol, mint a JSON, de különleges szintaxisa miatt nagyon rugalmasan lehet használni: beágyazásokat szóközökkel lehet jelölni, a Python-hoz hasonlóan, de ezek mellett ugyanúgy lehet a JSON formátumát is használni, akár keverni a kettőt is. Tömböket hozhatunk létre úgy, hogy minden egyes elemét kötőjelekkel kezdve megadjuk, és ezeket objektumokkal kombinálni asszociatív tömböket is létre tudunk hozni. A szövegeket nem szükséges idézőjelek közé tenni, és dupla felkiáltó jellel címkéket is hozzá lehet adni az értékekhez. [26]
A YAML-t a projektemben a szerver program konfigurációjának megadására fogom használni, mert szerintem az egyszerűbb szintaxisával letisztultabb konfigurációs fájlt lehet készíteni, és mivel a YAML megjegyzéseket is támogat, ezért a különböző konfigurációs értékekhez rövid magyarázatot vagy útmutatást is meg tudok adni.

6 Implementáció

6.1 Felhasznált eszközök

Az implementálás elkezdése előtt kitűztem magamnak azt a célt, hogy az alkalmazás programozásához csak a gyári dokumentációkat, mintakódokat használjam fel, a harmadik forrásból és mesterséges intelligenciától származó kódokat pedig nem. A backend programozásához a Go kit mintaprojektjeire (https://gokit.io/examples/), illetve az implementáció során használt csomagok dokumentációját használtam fel. A Go nyelv használatát a Go by examples (https://gobyexample.com/) nevű oldal példáinak kipróbálásával gyakoroltam. A frontend készítése során az MDN Web Docsra (https://developer.mozilla.org/en-US/), a Bootstrap (https://getbootstrap.com/docs/5.3/), a

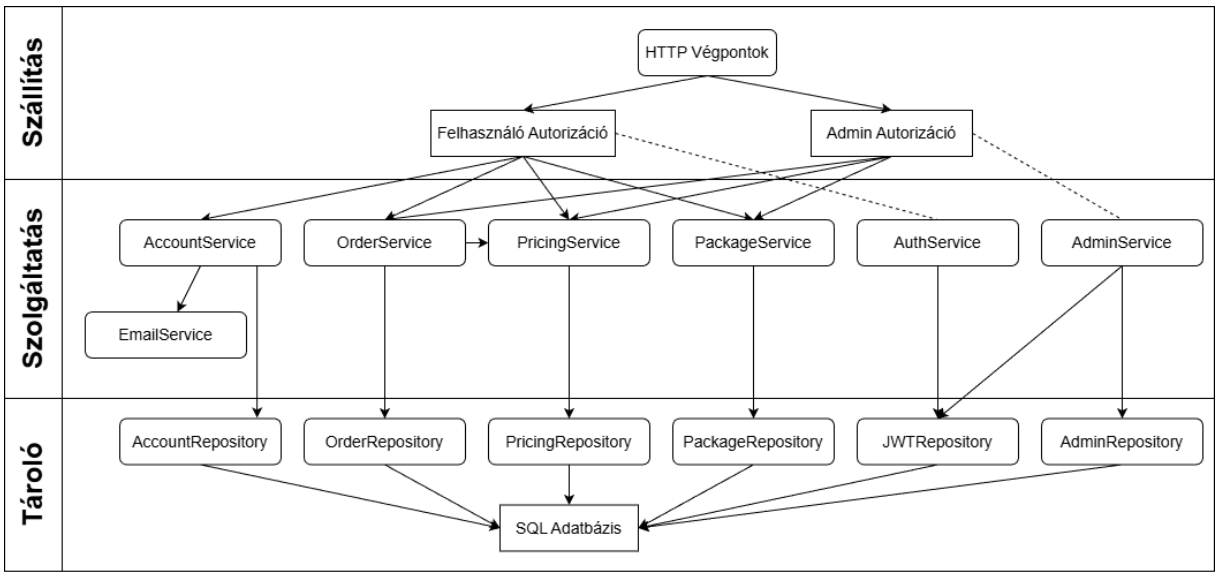
Svelte (https://svelte.dev/docs/svelte/) és a SvelteKit (https://svelte.dev/docs/kit/) dokumentációira hagyatkoztam.

A fejlesztés során többféle fejlesztői eszközt is alkalmaztam. A backend programozását a JetBrains által fejlesztett GoLand nevű IDE-ben készítettem, a HTTP API végpontokat pedig Postman-nel teszteltem. Az adatbázis kezeléséhez a pgAdmin 4 programot használtam. A frontend programozását Visual Studio Code-dal csináltam, és az npm csomagkezelőt használtam a SvelteKit-hez szükséges fejlesztői környezet felállításához. A kétnyelvű odalak elkészítését a Sherlock nevű VSCode bővítmény segítette, amivel gyorsabban lehetett a fordításokat elhelyezni. A frontend debugolásához és weboldal dizájnok tökéletesítéséhez a Chrome Dev Tools is nagy segítség volt.

6.2 Backend

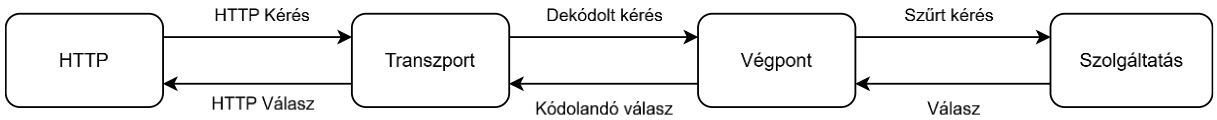
6.2.1 Backend felépítése

A backend fejlesztése során a Go kit architekturális mentalitását követve a backend funkcióit szolgáltatásokra osztottam fel (2. Ábra): az AccountService a felhasználói fiókok, az OrderService a rendelések, a PackageService a csomagok és az AdminService az adminisztrációs fiókok kezelésért felelős. A PricingService számolja ki az OrderService és az Árkalkulátor űrlap számára a csomagszállítások árait. Az AuthService és az AdminService feladata, hogy a beérkező felhasználói és adminisztrátori hitelesítő adatokat érvényesítse. A szolgáltatásokhoz tartozó tárolók perzisztálják a szolgáltatások adatait.



2. ábra: Backend felépítése

Minden szolgáltatás rendelkezik transzportokkal és végpontokkal, amiken keresztül lehet elérni őket HTTP-n keresztül. A transzportok feladata, hogy beérkező HTTP kérést dekódolja az alkalmazás számára, illetve a backend viszontválaszát továbbítsa. A végpontok feladata a dekódolt kérések előfeldolgozása, hozzáférési jogosultságok ellenőrzése. Kivételt képez az EmailService, amelynek a feladata az e-mailek kiküldése, amelyhez csak az AccountService fér hozzá közvetlenül fiók megerősítő e-mailek küldéséhez.



3. ábra: HTTP Kérések feldolgozásának folyamata

A projekt kódbázisa két fő részből áll: a main.go fájlból, amely összekapcsolja a backend különböző szolgáltatásait, végpontjait és transzportjait, és a services könyvtárban található szolgáltatás és segédfüggőségekből. A services könyvtár felépítését a program egységeinek bemutatása közben fogom taglalni.

6.2.2 Szolgáltatások felépítése

Az összes szolgáltatás (az EmailService kivételével) a három részből épül fel: az üzleti logika implementációjából, egy naplózó és egy metrikákat gyűjtő middlewarekből. Minden szolgáltatás definiál egy egyedi interfészt, amelyet a middlewareknek is meg kell valósítaniuk. Ezen interfészek mentén lehet a middlewarek-et a fő üzleti logikával összekapcsolni a díszítő programozási minta használatával. Minden szolgáltatás egy saját könyvtárban helyezkedik el a /services mappán belül, a fő üzleti logika implementációja pedig a könyvtárral azonos nevű .go fájlban találhatóak.

A következő fejezetekben az AccountService Verify metódusán keresztül fogom bemutatni, hogyan néznek ki a szolgáltatások a gyakorlatban. Ahogyan a 4. ábrán is látható, az AccountService interfésze tartalmazza Verify metódus definícióját. Ez a metódus a felhasználói fiókok aktiválásáért felel, bemenetként az aktiválandó fiók e-mail címét és az aktiváló kódot kéri, válaszként csak az esetleges hibákat adja vissza.

```go
type Service interface {  9 usages  3 implementations
    Register(email string, password string, name string, phoneNumber string) error
    Get(email string) (*models.Account, error)  3 implementations
    Verify(email string, code string) error  3 implementations
    Update(account models.Account) error  3 implementations
    Delete(email string) error  3 implementations
}
```

4. ábra: AccountService szolgáltatás interfésze (/services/account/account.go)

Az AccountServicehez tartozó függőségek a /services/account könyvtárban találhatóak. Az account.go fájl tartalmazza az interfészdefiníciókat, és a konkrét üzleti logika implementációját. A Verify metódus (5. ábra) leellenőrzi, hogy a betölti a megadott e-mail címhez tartozó felhasználói fiókot, leellenőrzi, hogy a megadott aktiválókód érvényes-e, majd eltárolja az aktiválás időpontját.

```go
func (s service) Verify(email string, code string) error {
    account, err := s.Repo.FindByEmail(email)
    if err != nil { return err }

    if *account.VerificationID != code { return ErrInvalidVerificationCode }
    t := time.Now()
    account.VerifiedAt = &t
    err = s.Repo.Store(account)
    if err != nil { return err }

    return nil
}
```

5. ábra: AccountService Verify implementációja (/services/account/account.go)

A metrikák begyűjtését megoldó middleware-k mindig az adott szolgáltatáshoz tartozó instrumenting.go fájlban találhatóak. Ezekbe a middlewarek két Prometheus metrika gyűjtésére alkalmasak, amelyek main függvényben vannak definiálva. (6. ábra) A requestCount a kiszolgált kérések darabszámát, a requestLatency a kiszolgált kérések válaszidejét méri.

```go
fieldKeys := []string{"method", "error"}
promNamespace := "package_app"
requestCount := func(subsystem string) *kitprometheus.Counter {
    return kitprometheus.NewCounterFrom(stdprometheus.CounterOpts{
        Namespace: promNamespace,
        Subsystem: subsystem,
        Name:      "request_count",
        Help:      "Number of requests received.",
    }, fieldKeys)
}
requestLatency := func(subsystem string) *kitprometheus.Histogram {
    return kitprometheus.NewHistogramFrom(stdprometheus.HistogramOpts{
        Namespace: promNamespace,
        Subsystem: subsystem,
        Name:      "request_latency_microseconds",
        Help:      "Total duration of requests in microseconds.",
    }, fieldKeys)
}
```

6. ábra: Prometheus metrikák definíciója (main.go)

Az egyedi InstrumentingMiddleware implementációk során ezeket a metrikákat növeljük a megfelelő értékekkel, illetve egyedi fejlécekkel. A Verify megvalósítása (7. ábra) során a fejlécekhez a metódus nevét, és az esetleges hibát adjuk hozzá, a RequestCountot növeljük eggyel és a RequestLatencynek az eltelt időt állítjuk be.

```go
func (mw InstrumentingMiddleware) Verify(email string, code string) (err error) {
    defer func(begin time.Time) {
        lvs := []string{"method", "Verify", "error", fmt.Sprint(err != nil)}
        mw.RequestCount.With(lvs...).Add( delta: 1)
        mw.RequestLatency.With(lvs...).Observe(time.Since(begin).Seconds())
    }(time.Now())

    err = mw.Next.Verify(email, code)
    return err
}
```

7. ábra: InstrumentingMiddleware Verify implementációja (/services/account/instrumenting.go)

A naplózó middlewarek az adott szolgáltatáshoz tartozó logging.go fájlban találhatóak. Feladatuk, hogy minden egyes kérést naplózzanak a sztenderd kimenetre kulcs-érték párok formájában. Ehhez a main függvényben definiált logger objektumot használják fel, amely a megadott kulcs-érték párokat automatikus formázza és kiírja a konzolra. (8. ábra)

```
logger := log.NewLogfmtLogger(os.Stdout)
```

8. ábra: Sztenderd kimenetre naplózás definíciója (main.go)

A naplózó middlewarek A különböző LoggingMiddleware implementációs tetszőleges értékeket tudnak naplózni. A Verify implementáció esetében a metódus nevét, a hibát és az eltelt időt naplózza a rendszer. (9. ábra)

```
func (mw LoggingMiddleware) Verify(email string, code string) (err error) {
    defer func(begin time.Time) {
        _ = mw.Logger.Log(
            keyvals...: "method", "verify",
            "err", err,
            "took", time.Since(begin),
        )
    }(time.Now())

    err = mw.Next.Verify(email, code)
    return
}
```

9. ábra: LoggingMiddleware Verify implementációja (/services/account/logging.go)

Az elkészült programrészek a main függvényben vannak összefűzve egy szolgáltatássá, ahol a különböző függőségek, például tárolók, szolgáltatások, konfigurációk, illetve a naplózó és Prometheus metrikák objektumai is kapcsolva vannak a szolgáltatáshoz. Az így összerakott szolgáltatás már alkalmas arra, hogy végpontokat adjunk hozzá.

```go
var accountService account.Service
{
    accountService = account.CreateAccountService(accountRepository, emailService, frontendLocation)
    accountService = account.LoggingMiddleware{Logger: logger, Next: accountService}
    accountService = account.InstrumentingMiddleware{
        RequestCount:   requestCount( subsystem: "account_service"),
        RequestLatency: requestLatency( subsystem: "account_service"),
        Next:           accountService,
    }
}
```

10. ábra: AccountService felépítése (main.go)

6.2.3 Végpontok felépítése

A végpontok létrehozásának fő célja, hogy a szolgáltatásokat egy absztrakt interfészen keresztül lehessen elérni. Egy végpontot egy Endpoint típusú függvény megvalósításával lehet létrehozni, amely bemenetként a futási kontextust és egy kérés interfészt kér, és egy válasz interfészt és a felmerülő hibákat adja vissza. Végpontok implementálására többféle lehetőségünk is van: lehetséges a kontextus és a kérés alapján a szolgáltatás összes metódusát egy végponton ellátni, de én azt a megoldást választottam, hogy minden egyes metódushoz egy külön végpontot készítettem. Az összes végpontot closure technikával definiáltam, így a használatuk sokkal kényelmesebbé válik. A Verify metódushoz a MakeVerifyEndpoint függvény készít egy végpontot. (11. ábra) Ez a végpont a kérés interfészt egy VerifyRequestként értelmezi, majd az értékeivel meghívja a Verify metódust.

```go
func MakeVerifyEndpoint(s Service) endpoint.Endpoint {  1 usage
    return func(ctx context.Context, request interface{}) (interface{}, error) {
        req := request.(VerifyRequest)
        err := s.Verify(req.Email, req.Code)
        return nil, err
    }
}
```

11. ábra: Verify metódus végpontja (/services/account/transport.go)

6.2.4 Transzportok felépítése

A transzportok fő feladata, hogy a HTTP kéréseket összekapcsolja a végpontokkal. Ehhez a beérkező HTTP üzeneteket dekódolja egy egyedi dekódolási logika segítségével, amivel a

különböző helyen található információk, például URL keresési paraméterek, a kérés fejlécei és törzse elhelyezhetőek a végpont kontextusában vagy kérési interfészében.
Az alkalmazásom során a HTTP kéréseknek két formátuma van: a kért információt a keresési paraméterrel, vagy a kérés törzsével kell megadni. A kérések és válaszok törzse mindig JSON formátumban vannak megadva. A Verify metódus HTTP transzport definíciója (13. ábra) a következőféleképpen épül fel: a MakeVerifyEndpoint metódussal megadjuk a transzporthoz hozzárendelni kívánt végpontot, beállítjuk, hogy a kérést a DecodeVerifyRequest (12. ábra) segítségével a kérés törzséből olvassuk ki, és a választ és a hibaüzeneteket is JSON formátumban legyenek továbbítsuk. Az így elkészült végpont main függvény legvégén hozzá van rendelve egy HTTP Listenerhez, amely /verify útvonalon POST metódussal várja a http kéréseket.

```
func DecodeVerifyRequest(_ context.Context, r *http.Request) (interface{}, error) {  1 usage
    var request VerifyRequest
    if err := json.NewDecoder(r.Body).Decode(&request); err != nil { return nil, err }
    return request, nil
}
```

12. Ábra: DecodeVerifyRequest dekódoló metódus (/services/account/transport.go)

```
verifyHandler := httptransport.NewServer(
    account.MakeVerifyEndpoint(accountService),
    account.DecodeVerifyRequest,
    EncodeResponse,
    httptransport.ServerErrorEncoder(JSONErrorEncoder),
)
```

13. ábra: Verify metódus transzportja (main.go)

6.2.5 Tárolók felépítése

Az alkalmazásban szolgáltatások az adatok tárolását, módosítását egy tároló interfészen keresztül végzik, így a konkrét perzisztálási implementáció el van választva a szolgáltatás logikájától. A tárolókat úgy terveztem meg, hogy az adatok tárolása és módosítása is közös Store metóduson keresztül történnek. A tároló implementációk mindegyike a GORM-ot használja az adatbázis utasítások kiadására. A tárolók kódbázisa a /services/repository

könyvtárban található. Ebben a fejezetben a tárolók működését a PackageService és Repository-n keresztül fogom bemutatni. A PackageService definiál egy tároló interfészt (14. ábra), amelyet a PackageRepository teljesen implementál.

```
type Repository interface {  2 usages  1 implementa
    Store(*models.Package) error  1 implementatior
    Find(id string) (*models.Package, error)
    Delete(id string) error  1 implementation
}
```

14. ábra: Package szolgáltatás Repository interfésze (/services/package/package.go)

A tárolók és a szolgáltatások közötti adatcsere modellek használatával történik, amelyeket a /services/models könyvtárban lehet megtalálni. Ezek a modellek magukba ágyazzák a gorm.Model struktúrát és GORM működését szabályzó tageket is tartalmaznak. Ez látható a Package modelljében is. (15. ábra)

```
type Package struct {  19 usages
    gorm.Model
    PackageID string   `gorm:"not null; unique"`
    Length    int      `gorm:"not null"`
    Width     int      `gorm:"not null"`
    Height    int      `gorm:"not null"`
    Price     int      `gorm:"not null"`
    From      Location `gorm:"embedded;embeddedPrefix:from_"`
    To        Location `gorm:"embedded;embeddedPrefix:to_"`
    OrderID   string   `gorm:"not null"`
    Statuses  []Status `gorm:"foreignKey:PackageID;references:PackageID"`
}
```

15. ábra: Package modell (/services/models/package.go)

A tárolók megvalósítása során a GORM-ot a generikus interfészén keresztül alkalmaztam. Például a Find metódus implementációjában (16. ábra) a generikusként a Package modell van megadva. A lekérés során a megadott azonosítójú csomagot keressük, és a találatba a csomaghoz tartozó státuszokat is betöltjük az idegen táblákból.

```go
func (p PackageRepository) Find(id string) (*models.Package, error) {
    ctx := context.Background()
    pack, err := gorm.G[models.Package](p.Db).
        Where( query: "package_id = ?", id).
        Preload( association: "Statuses",  query: nil).First(ctx)
    return &pack, err
}
```

16. ábra: PackageRepository Find metódus implementációja (/services/repository)

6.2.6 Árazás kiszámítása

Az csomagok kiszállításának díját a következő képlettel számolja ki a program: alapdíj + egészre_kerekítés(távolság * km_díj / 50) * 50, ahol az alapdíjat és a km_díjat a szolgáltatás tárolt értékekként kapja. A távolság a feladó és címzett közötti földrajzi távolság kilométerben megadva. A távolság kiszámításához meg kellett oldanom, hogy egy megadott közigazgatási címhez egy földrajzi koordinátát lehessen rendelni. Ezt a folyamatot geokódolásnak hívjuk. Egy geokódolási szolgáltatás elkészítése bonyolult folyamat, ezért egy külső, ingyenes megoldást kerestem az interneten. Így találtam rá Geocoding API (https://geocode.maps.co/) nevű HTTP API-ra. Az API használatakor egy másik probléma is felmerült, az API ingyenes csomagjában csak másodpercenként egy kérés engedélyezett, ezért a golang.org/x/time/rate csomag segítségével limitálnom kellett a lekérdezések gyakoriságát. Az így megszerzett koordináták közti koordináták kiszámításakor oda kellett arra is figyelnem, hogy a Föld görbülete miatt a távolságot nem lehet szimpla geometria módszerekkel kiszámolni. A számítás helyes elvégzéséhez Vincenty képletét alkalmaztam a *github.com/jftuga/geodist* csomag segítségével.

6.2.7 Autentikáció és autorizáció

A felhasználók és adminisztrációk API végpontjai között is vannak olyanok, amelyeket csak azonosítás után lehet elérni. A felhasználók az fiókjuk e-mailjével és jelszójával, az adminisztrátorok egy felhasználónév jelszó párral tudják azonosítani magukat a /login és /admin/login útvonalokon keresztül, és válaszként egy hozzáférési és frissítési JWT-t kapnak. Ezek JWT-k digitálisan alá vannak írva HS256 módszerrel, és regisztrált állítások közül a Subject mezőben a felhasználó azonosítóját, JWT ID mezőben egy UUID azonosítót, az Audience mezőben a felhasználó (admin/user) és token (access/refresh) fajtáját és egy érvényességi időt tartalmaznak. A frissítési tokennel lehetséges új hozzáférési tokent szerezni

anélkül, hogy újra be kellene jelentkezni. A JWT-t az autentikációt igénylő végpont elérése során a HTTP Kérés Authorization fejlécébe meg kell adni Bearer típussal. A JWT-k használata egyéb előnyökkel is jár, például a /getall útvonalon a végpont a JWT állításaiból következteti ki, hogy melyik felhasználóval kapcsolatos adatait kell kikérnie a GetAllOrder metódusnak.

```
func MakeGetAllOrdersRequest(s Service) endpoint.Endpoint {  1 usage
    return func(ctx context.Context, request interface{}) (interface{}, error) {
        claims := ctx.Value(jwt.JWTClaimsContextKey).(*stdjwt.RegisteredClaims)
        orders, err := s.GetAllOrdersForUser(claims.Subject)
        if err != nil { return nil, err }
        var response []OrderDTO
        response = make([]OrderDTO, len(*orders))
        for i, order := range *orders {
            response[i] = ToOrderDTO(order)
        }
        return response, nil
    }
}
```

17. ábra: MakeGetAllOrdersRequest végpont (/services/order/transport.go)

A JWT-k érvénytelenítéséhez az JWT-k a JWTRepositoryban vannak eltárolva. A go kit JWT feldolgozó algoritmusát kibővítettem egy ellenőrző funkcióval, amely ellenőrzi, hogy az adott JWT nincs-e érvénytelenítve (kitörölve az adatbázisból). (18. ábra)

```
id := token.Claims.(*stdjwt.RegisteredClaims).ID
_, err = jwtRepository.Find(id)
if errors.Is(err, gorm.ErrRecordNotFound) {
    return  response: nil, jwt.ErrTokenExpired
}
```

18. ábra: JWT-k érvényességét ellenőrző kódrészlet (main.go, JWTParser metódus)

6.2.8 Egyéb implementációs kérdések

Az AccountService egy fiók regisztrációjakor egy e-mailt küld a felhasználó e-mail címére. Ehhez a *gopkg.in/gomail.v2* csomag segítségével létrehoz egy Message objektumot, amelyet továbbad az EmailService szolgáltatásnak. Az EmailService ezután csatlakozik egy SMTP

szerverhez, melyen keresztül elküldi az e-mailt. Mivel egy élesben is használható SMTP szerver konfigurációja összetett feladat, ezért egy ingyenesen elérhető SMTP szolgáltatást, az SMTP2GO-t (https://smtp2go.com) használtam.

A backend sok részét nem lehet alapértelmezett konfigurációkkal megoldani, például adatbáziskapcsolat megadása, ezért egy külső konfigurációs állomány beolvasását is meg kellett oldanom. Ehhez a YAML fájlformátumot választottam, és a *gopkg.in/yaml.v3* csomagot használtam fel a konfigurációs fájl parszolására. A konfiguráció ellenőrzését és feldolgozását végző kódbázis a /services/config/config.go fájlban található.

A backend fejlesztése során több objektumhoz is szükséges volt, hogy egyedi azonosítót generáljak. A JWT azonosítók és a felhasználó hitelesítő kód generálásához UUID-t, a csomag és rendelés azonosítók készítéséhez pedig ULID-et alkalmaztam a *github.com/google/uuid* és *github.com/oklog/ulid/v2* csomagok segítségével.

Az alkalmazás működéséhez sok HTTP API végpontot kellett készítenem, amelyek teljes listája megtalálható a függelékben.

6.3 Frontend

6.3.1 Frontend felépítése

A frontend SvelteKit projektje következőképpen épül fel: az /src/routes könyvtár tartalmazza a böngészőből álló útvonalakat. Minden weboldal elérési útvonalát egy mappa jelöli, ami mindig egy +page.svelte és egy +page.server.ts fájlokat tartalmaz, amelyek az adott oldal felhasználói felületét, és szerver oldali tevékenységét adják meg. A /static útvonal alatt találhatóak a statikus tartalmak. Az /srcben található *app.html* oldal az alkalmazás kiindulási pontja, az útvonalak feloldása során ebbe az oldalba ágyazza bele a SvelteKit dinamikus tartalmakat, ezért a külső CCS és JavaScript hivatkozásokat itt érdemes hozzáadni a projekthez. Az ugyanitt található +layout.svelte fájlban található az összes oldalon látható navigációs sáv kódja. Az útvonalakat két nagy részre lehet osztani tartalmi szempontból: /app alatt található oldalak a felhasználói felületet, a /admin aloldalai az adminisztrációs felületet alkotják.

6.3.2 Paraglide

A weboldal tervezésénél fontos szempont volt, hogy az oldalak kétnyelvűek lehessenek. Ehhez a Paraglide JS nagy hasznát vettem. A */messages* könyvtárban található *en.json* és *hu.json* fájlok

tartalmazzák a fordításokat, amelyek alapján a Paraglide fordítóprogramja készíti el a szkriptekben és weboldalokon meghívható függvényeket. A Paraglide biztosítja a getLocale és setLocale függvényeket, amelyek segítségével lekérhető és beállítható az aktuális oldalnyelv. Ezeket a függvényeket hozzárendeltem az /src/routes/+layout.svelte fájlban található fejléc nyelvállító gombjaihoz (19. ábra), az többi oldalon pedig a szükséges helyeken elhelyeztem a fordításokat meghívó függvényeket is.

```
<NavItem>
  <NavLink
    class="pe-1 {getLocale() == 'hu' ? 'active' : ''}"
    on:click={() => setLocale("hu")}>HU</NavLink
  >
</NavItem>
<NavItem>
  <NavLink
    class={getLocale() == "en" ? "active" : ""}
    on:click={() => setLocale("en")}>EN</NavLink
  >
</NavItem>
```

19. ábra: Nyelvállító gombok a fejlécben (/src/routes+layout.svelte)

6.3.3 Actions

A webalkalmazásban található összes űrlap feldolgozását és az API felé való tovább küldését az Actions szerver oldali típus implementálásával készítettem el. Amikor a felhasználó lead egy űrlapot, akkor azt az ahhoz tartozó Actions feldolgozóegység elkapja. A feldolgozás során hozzá lehet férni a leadott űrlapadatokhoz, ezeket utána szükség szerint validáltam és utána továbbítottam a backend API felé JSON formátumban. (20. ábra)

```
export const actions: Actions = {
    default: async (event) => {
        const formData = await event.request.formData();

        const order = createOrderFromFormData(formData)
        if (isActionFailure(order)) {
            return order
        }
        const body = JSON.stringify(order)

        try {
            const response = await event.fetch(apiLocation + "/admin/order", {
                method: "POST",
                body: body,
                headers: {
                    "Content-Type": "application/json",
                    "Authorization": "Bearer " + event.cookies.get("access")
                },
            })
            const data = await response.json()
            if (!response.ok) {
                if (data.error) {
                    return fail(response.status, { error: data.error })
                }
                return fail(response.status, { error: "Internal Error" })
            }
            return { success: true }
        }
        catch (e) {
            return fail(500, { error: "Internal error: " + e })
        }
    }

}
```

20. ábra: Csomag adatainak frissítése Actions-el (/src/routes/admin/order/+page.server.ts)

Az API válaszától függően az adott Actions hibát vagy a kért értékeket adja vissza, amelyet az kliens oldalon a *$props* rúna segítésével a form objektumból ki lehet olvasni. A hibát vagy a sikerességet Bootstrap Alert komponensek segítségével jeleníti meg az alkalmazás. (21. ábra) Egy Actions-ben több feldolgozóegységet is el lehet helyezni, amelyeket egyedi űrlapakciók megadásával lehet meghívni.

```
{#if form?.error}
    <Alert class="container content" dismissible id="error" color="danger">
        <h4 class="alert-heading text-capitalize">{m.error()}</h4>
        {form.error}
    </Alert>
{:else if form?.success}
    <Alert class="container content" dismissible color="success">
        <h4 class="alert-heading text-capitalize">{m.success()}</h4>
    </Alert>
{/if}
```

21. ábra: Hiba és siker üzenetek kódja (/src/routes/admin/order/+page.svelte)

6.3.4 SvelteStrap

A SvelteStrap egy Svelte komponens könyvtár, amellyel Bootstrap komponenseket lehet elhelyezni Svelte komponensekként. Használatának előnye, hogy a kód olvashatóbbá egyszerűbbé válik, mert nem kell a Bootstrapre jellemzően az osztályattribútumokat többszörösen ismételni. (22. és 23. ábra) A SvelteStrap hasznát legtöbbször a lebegő címkés beviteli mezők készítéséhez vettem. (24. ábra) Hátránya, hogy ezeknek a komponensek a stílusa nem teljesen testreszabhatóak, ezért sok esetben kényszerültem natív Bootstrap használatára.

```
<FormGroup floating>
    <Input
        disabled={submitInProgress}
        name="zip"
        bind:value={data.order.ZIPCode}
        required
    />
    <div slot="label">
        {m.postcode()} <span class="text-danger">*</span>
    </div>
</FormGroup>
```

22. ábra: Lebegő címkés beviteli mező kódja SvelteStrap használatával

```
<div class="form-floating mb-3">
    <input
        disabled={submitInProgress}
        name="zip"
        bind:value={data.order.ZIPCode}
        required
    />
    <label for="zip">{m.postcode()} <span class="text-danger">*</label>
</div>
```

23. ábra: Lebegő címkés beviteli mező kódja SvelteStrap nélkül

24. ábra: Lebegő címkés beviteli mező kinézete a böngészőben, kijelöletlen és kijelölt állapotban

6.3.5 Kampók

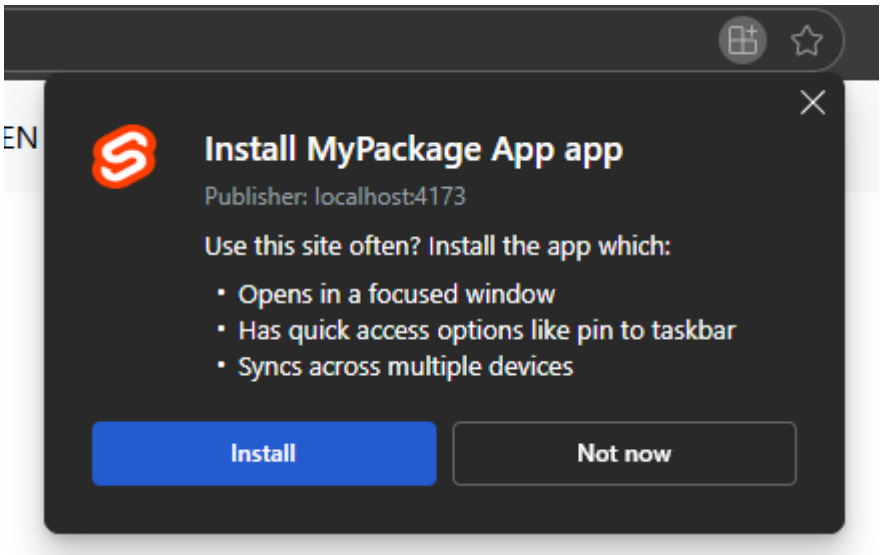
Ahogyan azt már említettem, kampók használatával el lehet kapni a webszerveren áthaladó forgalmat, és tetszőleges logikával módosítani lehet azt. Az én alkalmazásomban csak szerver oldali kampót használtam, amelyet az /src/hooks.server.ts fájlban lehet megtalálni. A kampó implementációm három részből áll: az adminisztrátori autorizációból, a felhasználói autorizációból és a Paraglide middlewarejéből, amely a fordítás működéséhez szükséges. Az adminisztrátori és a felhasználói autorizáció is három nagyon fontos feladatot lát el: frissítik a lejárt hozzáférési JWT-ket, szabályozzák a védett oldalak elérését, és a locals mezőn keresztül biztosítják az Actions-ök számára az autorizációt igénylő API kérésekhez a hozzáférési JWT-t. Fontos megjegyezni, hogy az autentikációt nem a kampó végzi. A /app és /admin útvonalak alatt található Actions-ök meghívásával lehet be- és kijelentkezni, amely során a feldolgozó megváltoztatja az oldal sütijeit és locals mezőit. Az űrlap leadása után frissül az oldal, és átveszi a hitelesítő adatok kezelését a kampó.

6.3.6 PWA

Ahhoz, hogy a frontend által nyújtott weboldalakat PWA-ként lehessen használni, el kellett készítenem egy *manifest.json* fájlt, amely a statikus tartalmaknál található és egy link taggel az

app.htmlben is hivatkozva van. Ezek után, ha egy felhasználó megnyitja az oldalt a böngésző automatikusan felajánlja a felhasználónak, hogy telepítse az alkalmazást az eszközére. (25. ábra) Ez a lehetőség mobileszközökön és személyi számítógépeken is működik, de https protokoll használata nélkül csak a fejlesztő eszközön lehet kipróbálni.

Hogy a PWA offline állapotban is nyújthasson funkcionalitásokat, egy service worker-t is készítettem (/src/service-worker.ts). A SvelteKit ezt automatikusan hozzáadja a weboldalhoz. A service worker egy nagyon egyszerű szolgáltatást nyújt, ha nem érhető el az alkalmazás webszervere, akkor a böngésző hibaüzenete helyett betölti a helyi cache-ből az offline.html oldalt, ami felhívja a felhasználó figyelmét erre.



25. ábra: A Microsoft Edge böngésző automatikusan felajánlja az alkalmazás telepítését

6.3.7 Egyéb megoldások

Az űrlapadatok validálásához a *validator* nevű npm csomagot használtam fel, amely képes ellenőrizni a megadott telefonszámok, e-mailek, irányítószámok érvényességét is.

Szükségesnek éreztem egy olyan funkció bevezetését, hogy az űrlapok leadásakor és amíg az API-tól válaszra vár a weboldal ne lehessen módosítani a beviteli mezőket. Ezt úgy oldottam meg, hogy az űrlapokat tartalmazó oldalakon bevezettem egy *submitInProgress* nevű reaktív állapotot a *$state* rúna használatával, és ezt a reaktív állapotot hozzákapcsoltam az *input* tagek *disabled* értékéhez. A *form* tagekhez a SvelteKit-ben lehetőség van az *use:enhance* attribútum megadására, amellyel módosítható az űrlapok működése. (26. ábra) Az űrlapmezők letiltásához egy olyan függvényt kellett megadnom az attribútum értékének, amelyik először igazra állítja

a submitInProgress állapotot, és visszaad egy aszinkron függvényt, amely hamisra állítja azt. Ez az aszinkron függvény akkor fog lefutni, amikor az Actions befejezte a futását.

```
<form
    action=""
    method="post"
    class=""
    use:enhance={() => {
        submitInProgress = true;

        return async ({ update }) => {
            submitInProgress = false;
            await update({ reset: false, invalidateAll: true });
        };
    }}
>
```

26. ábra: *use:enhance* használata az űrlapok letiltásához (/src/routes/admin/order/+page.svelte)

Az összes böngészőből elérhető weboldalak listája megtalálható a függelékben.

7 Felhasználói útmutató

7.1 Beüzemelés

7.1.1 Szoftverek telepítése

A rendszerem fejlesztését és tesztelését Windows 11-en végeztem, ezért a beüzemelést is itt fogom bemutatni, de az általam használt technológiák, több platformon, például Linuxon és MacOS-en is elérhetőek, de a telepítésük merőben eltérhet. A weboldalak funkcionalitását Chromium alapú böngészőkben is teszteltem, így az olvasó számára is ilyen böngésző használatát javaslom.

A backend futtatásához a Go fordítóprogram szükséges. Ezt a Go weboldaláról (https://go.dev/dl/) lehet letölteni. A szakdolgozatom készítésekor a 1.25.4-es verziót használtam, de Go kereszt kompatibilitást ad az egyes verzió alverziói között is. A frontend beüzemeléséhez a Node.js telepítésére lesz szükség. A szükséges telepítőt a gyártó oldaláról (https://nodejs.org/en/download) lehet letölteni. A fejlesztés során 24 LTS verzióját

használtam. A Go-t és a Node.js-t is az alapértelmezett beállításokkal érdemes telepíteni. A PostgreSQL adatbáziskezelő rendszer telepítőjét a projekt weboldaláról (https://www.postgresql.org/download/) lehet letölteni. A telepítés során fontos, hogy a pgAdmin 4-et is telepítsük, illetve a superuser-nek megadott belépési adatait jegyezzük fel, mert ezek nélkül nem lehet az adatbázishoz hozzáférni a későbbiekben.
Opcionális lépésként a Prometheus szerverét is le lehet tölteni (https://prometheus.io/download/) ha a backend metrikáit is le szeretnénk kérdezni, amelyet nem kell telepíteni, mert csak egy hordozható futtatható állományból áll. A szoftverek telepítése után szükséges lehet újraindítani a számítógépet, hogy a parancssorból is elérjük ezeket.
Az alkalmazás beüzemeléséhez szükség lesz egy Git kliensre is, hogy hozzáférhessünk a forráskódhoz. Az alkalmazás forráskódját a projekt nyilvános Github tárolójából (https://github.com/a20th/packageapp-szakdolgozat) lehet kikérni.

7.1.2 Adatbázis beállítása

Az alkalmazás beüzemeléséhez szükséges egy adatbázis és egy ehhez az adatbázishoz hozzáférő felhasználót létrehozni, különben az alkalmazás backend része nem tudja hová az adatokat perzisztálni. Ezt legegyszerűbben a pgAdmin 4 menedzsment eszközzel lehetséges megoldani. A program automatikusan felismeri a helyi PostgreSQL telepítést, így indítás után csak be kell jelentkeznünk a superuser-ünk adataival. Ezután hozzunk létre egy új Login/Group role-t, ahol adjunk meg egy felhasználónevet, a Definition fülön egy jelszót, és a Priviliges fülön engedélyezzük, hogy ez a felhasználó be tudjon jelentkezni. Ezután hozzunk létre egy új Database-t. Fontos, hogy a létrejövő adatbázis tulajdonosának adjuk meg az új felhasználónkat (vagy manuálisan adjuk meg a szükséges jogosultságokat), mert különben a backend ORM-je nem fogja tudni létrehozni az adatbázis sémáját.

7.1.3 Backend beállítása

Ha kikértük az alkalmazás forráskódját, akkor a *back* mappában a parancssor segítségévek adjuk ki a *go build* parancsot. Ekkor meghívjuk az előbbiekben letöltött Go fordítóprogramot, ami először letölti az építési folyamathoz szükséges függőségeket, majd ugyanebben a mappában létrehozza a backend futtatható állományát back-go.exe néven.
A backend futtatása előtt létre kell hozni egy YAML formátumú konfigurációs állományt. Ebben a könyvtárban található egy config.example.yaml nevű fájl, amely egy mintaként szolgál

a saját konfiguráció létrehozásához. Az alkalmazás alapértelmezetten a saját könyvtárában config.yaml néven keresi a konfigurációs állományt, de a -c kapcsoló használatával megadható egy más helyen található YAML fájlt is.

A geocoding_api_key mezőhöz API kulcsot az https://geocode.maps.co/ oldalon lehet igényelni, amennyiben erre nincs lehetőségünk, a *pricing_dev* érték *true*-ra állításával az alkalmazás nem fogja a külső API-t használni az árak kiszámolásához. Az e-mail funkció működéséhez szükséges egy SMTP szerver megadása is, amihez az SMTP2GO (https://smtp2go.com) nevű ingyenes szolgáltatás használatát javaslom. Az oldalon való regisztráció után létre kell hozni egy SMTP User-t, és a Verified Senders oldalon a Single sender emails fülön hozzá kell adni egy létező e-mail címet, amivel a backend által készített e-maileket szeretnénk továbbítani. A megadott SMTP User adatait és a hitelesített e-mail címet adjuk meg a konfigurációban. Amennyiben nem szeretnénk az SMTP beüzemelésével bajlódni, állítsuk be az *email_dev* értéket *true*-ra, így a backend alkalmazás az elküldendő e-maileket a konzolra fogja kiíratni.

A *jwt_secret_key* értékét nem szükséges megváltoztatni, de internetes eszközök segítségével generálhatunk új aláíró kulcsot, ha szeretnénk.

A frontend hosztneve és portja az alapértelmezett értékeket tartalmazzák, ezek változtatni csak akkor lesz szükséges, ha frontendet más beállításokkal indítjuk el.

7.1.4 Frontend beállítása

A frontend beállításának első lépése, hogy ha megváltoztattuk a backend elérési útvonalát akkor a */frontend/config.ts* fájlban is be kell azt állítani. Ezután a frontend könyvtárában ki kell adni az *npm install* parancsot, amely letölti a projekthez szükséges függőségeket. Következő lépésként az npm run build parancsot, amely a build mappában létrehozza a node szerverrel futtatható állományokat. A buildet futtatni az npm run preview paranccsal tudjuk, ami alapértelmezetten a 4173-as porton nyitja meg az alkalmazást.

7.1.5 Prometheus beállítása

Ha a Prometheus szervert is szeretnénk használni, akkor először a *prometheus.yml* konfigurációt módosítani kell, hozzá kell adni egy új bejegyzést a *scrape_configs*hoz az alapértelmezés mintájára, ahol a *target*nek a backend elérésének kell lennie. Ezután futtathatjuk

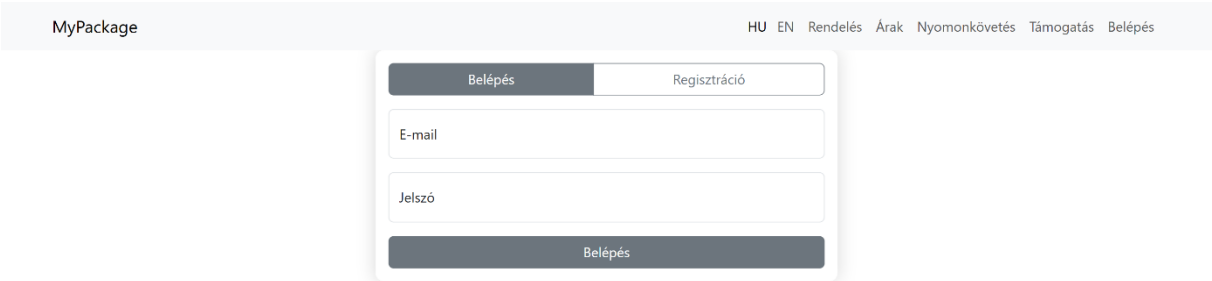
a Prometheust, és a böngészőből alapértelmezetten a *localhost:9090*-es címen érhetjük el a webes felületét.

7.2 Rendszer használata

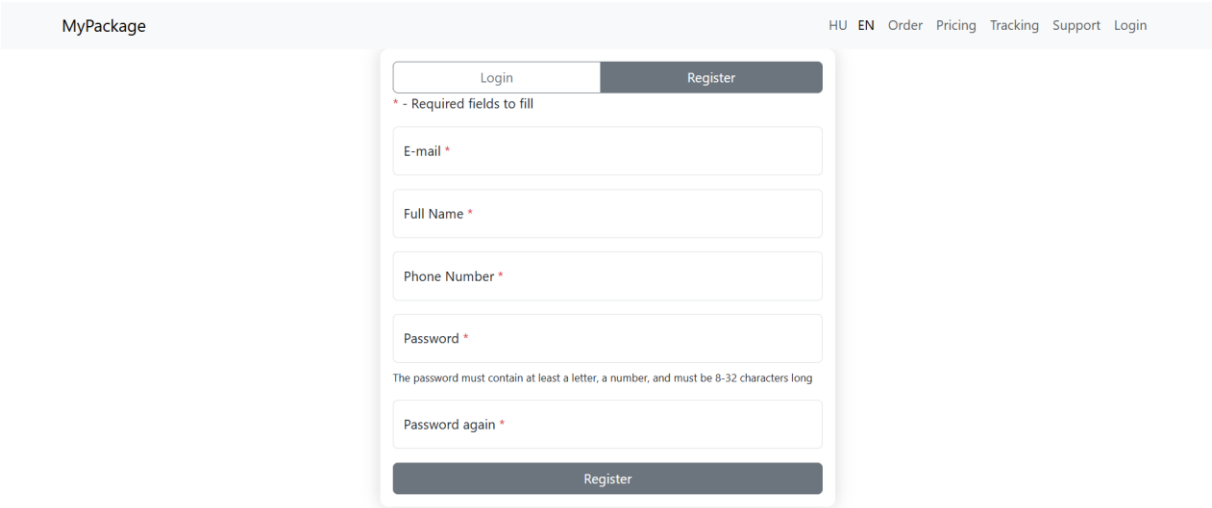
Ha sikeresen beüzemeltük az alkalmazást, akkor a böngészőben a frontend elérési útjára navigálva egy egyszerű weboldal fogadja a látogatókat, ahonnan lehetőség van belépni a felhasználói és adminisztrációs felületre.

7.2.1 Felhasználó felület

A felhasználói felület megnyitása után egy kombinált bejelentkező és regisztrációs felület fogadja a látogatót. (27. ábra) Amennyiben a felhasználó állítani szeretne az oldal nyelvén, azt a navigációs sávban található HU/EN gombokkal teheti meg. A regisztrációs felületen egy érvényes e-mail, teljes név, telefonszám és jelszó megadásával tud regisztrálni. (28. ábra) A regisztráció sikerességéről egy visszajelző ablak tájékoztatja a felhasználót, és sikeres regisztráció esetén kap egy e-mailt, amellyel aktiválni tudja a felhasználói fiókját.



27. ábra: Bejelentkező felület



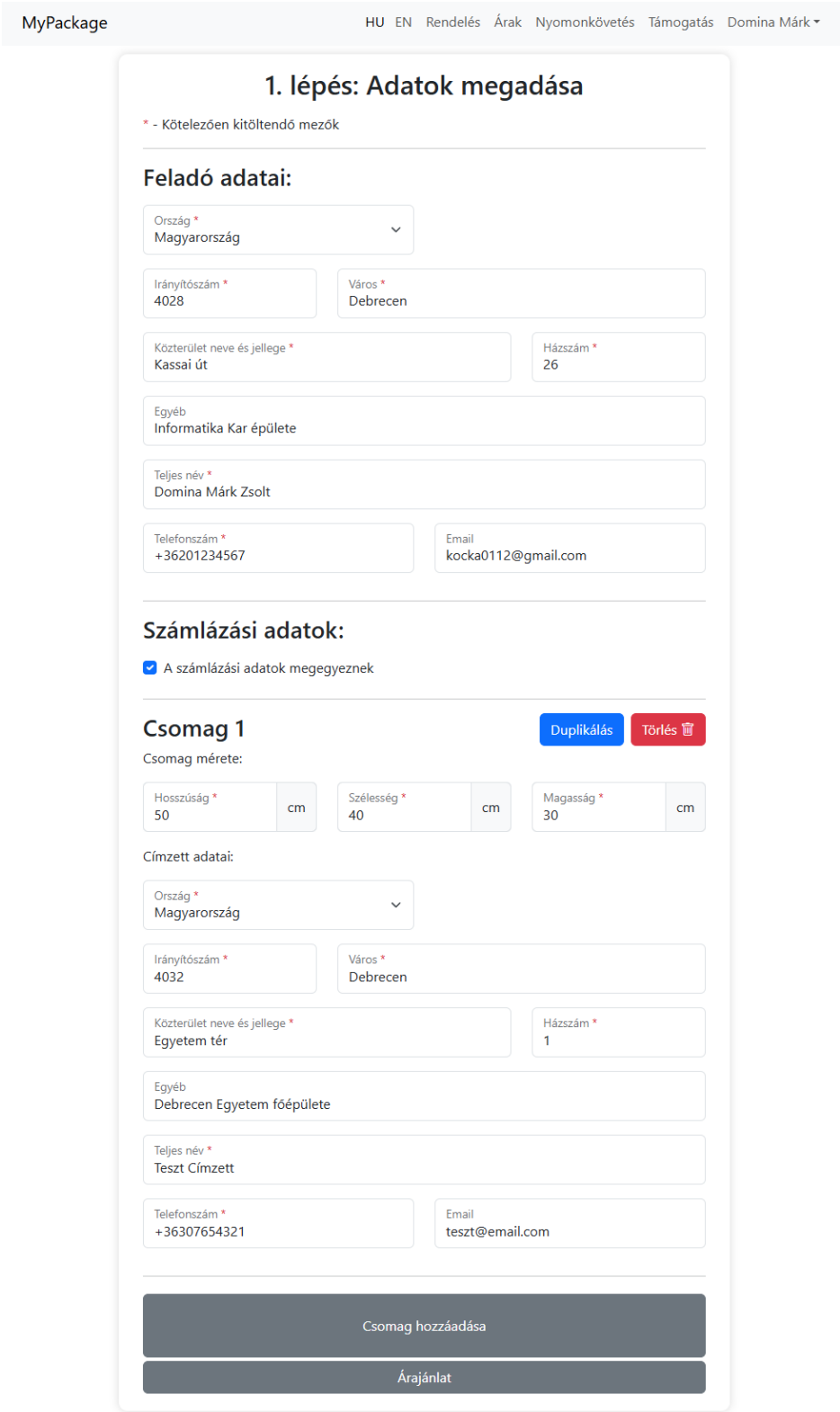
28. ábra: Regisztrációs felület

A felhasználói fiók aktiválása és bejelentkezés után a felhasználó a csomagmegrendelő felületen találja magát. Ezen az oldalon egy kiterjedt űrlap található, amelyen keresztül a rendeléshez szükséges információkat meg lehet adni. Lehetőség van arra, hogy a feladó adataitól eltérő számlázási adatokat adjon meg a felhasználó. Egyszerre több csomagot is lehet rögzíteni, és tetszőlegesen újat hozzáadni az űrlaphoz, megkettőzni vagy törölni is lehet ezeket. (29. ábra)

Ha a felhasználó elkészült az adatok megadásával, akkor az árajánlat gombra való kattintással tud árajánlatot kérni. Az árajánlat egy külön megerősítő felületen jelenik meg, ahol a csomagok kiszállításának árai tételekre le van osztva. A rendelés megerősítése gombbal el lehet fogadni az ajánlatot és leadni a megrendelést, vagy a vissza gombbal vissza lehet lépni az adatok megadásához.

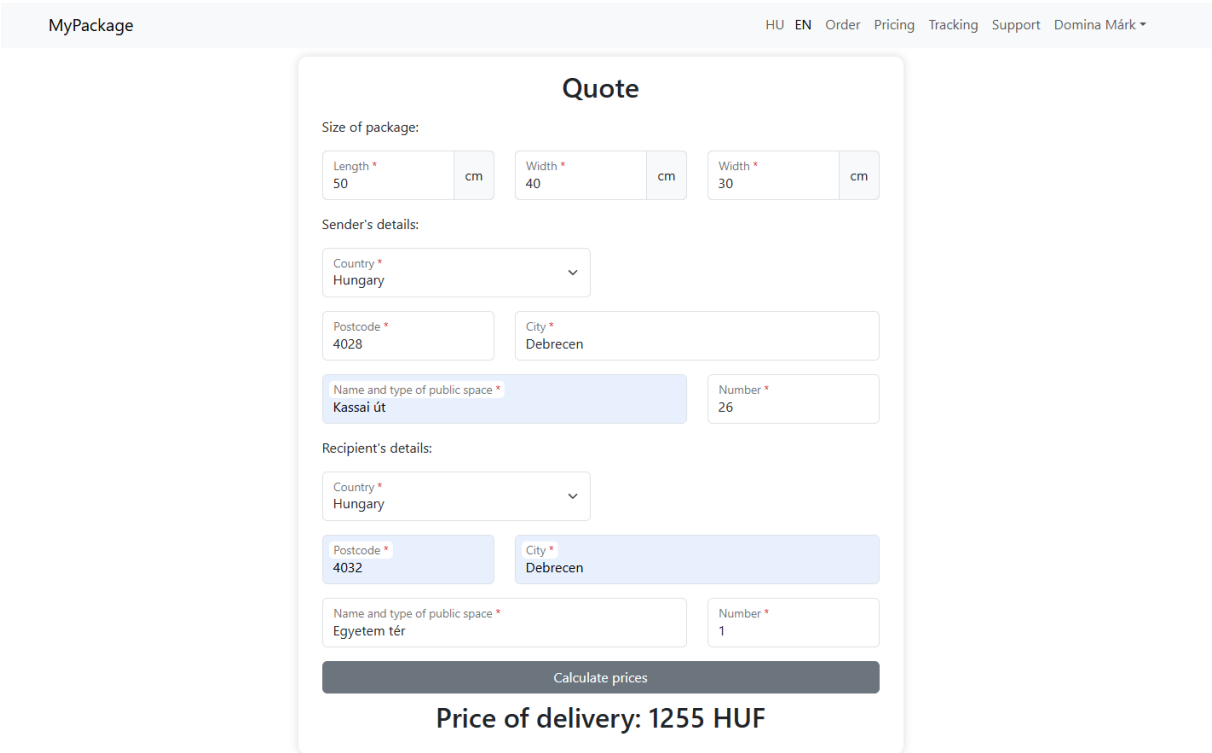
A felhasználók a fejlécből bejelentkezés nélkül is elérnek egy árkalkulátort, ahol a megrendelő felülethez képest kevesebb adat megadásával is tudnak árajánlatot kérni. Az űrlap kitöltése után az Árak kiszámítása gombra nyomva rövid töltés után megjelenik a csomag kiszállításának az ára. (31. ábra)

Ha a felhasználó be van jelentkezve, akkor a nevére való kattintással megjelenő lenyíló menüből meg tudja tekinteni az általa leadott rendeléseket. Ez az oldal lenyitható harmonikák listájából áll, ahol a harmonikák lenyitásával meg tudja tekinteni a rendelés és a hozzá tartozó csomagok adatait. (32. ábra) A követés nevű hiperhivatkozás átirányítja a felhasználót a csomaghoz tartozó csomagkövetési oldalra.

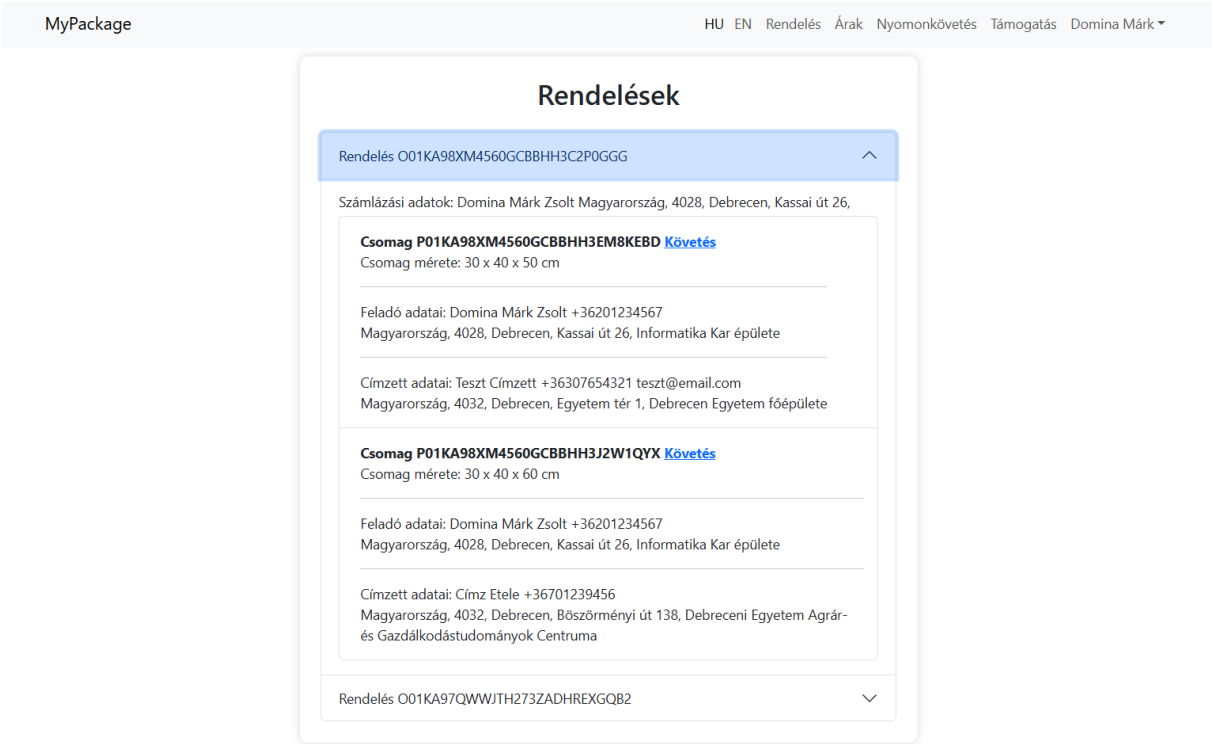


29. ábra: Csomagrendelés felülete

30. ábra: Rendelés megerősítés felülete

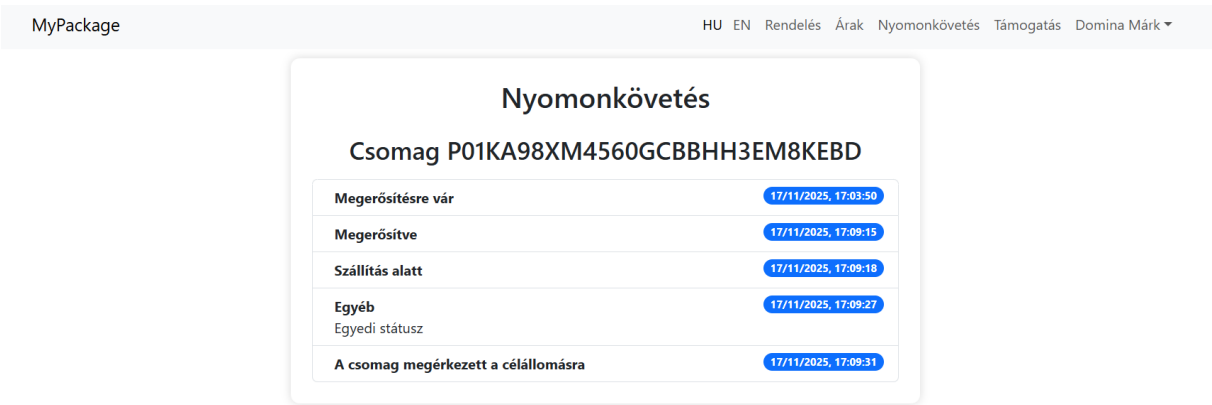


31. ábra: Árkalkulátor felülete



32. ábra: Felhasználó rendeléseit kilistázó felület

A nyomonkövetés oldalt nem csak a rendelésekből lehet elérni, hanem bejelentkezés nélkül a navigációs sávból is. Ekkor az oldal egy egyszerű űrlappal fogadja a felhasználót, amelyen a csomagazonosítójának megadásával meg lehet tekinteni a csomaghoz rögzített állapotokat. Az állapotok listájában látható az állapot neve, rögzítésének időpontja, és esetenként egy megjegyzés is. (33. ábra)



33. ábra: Nyomonkövetés felülete

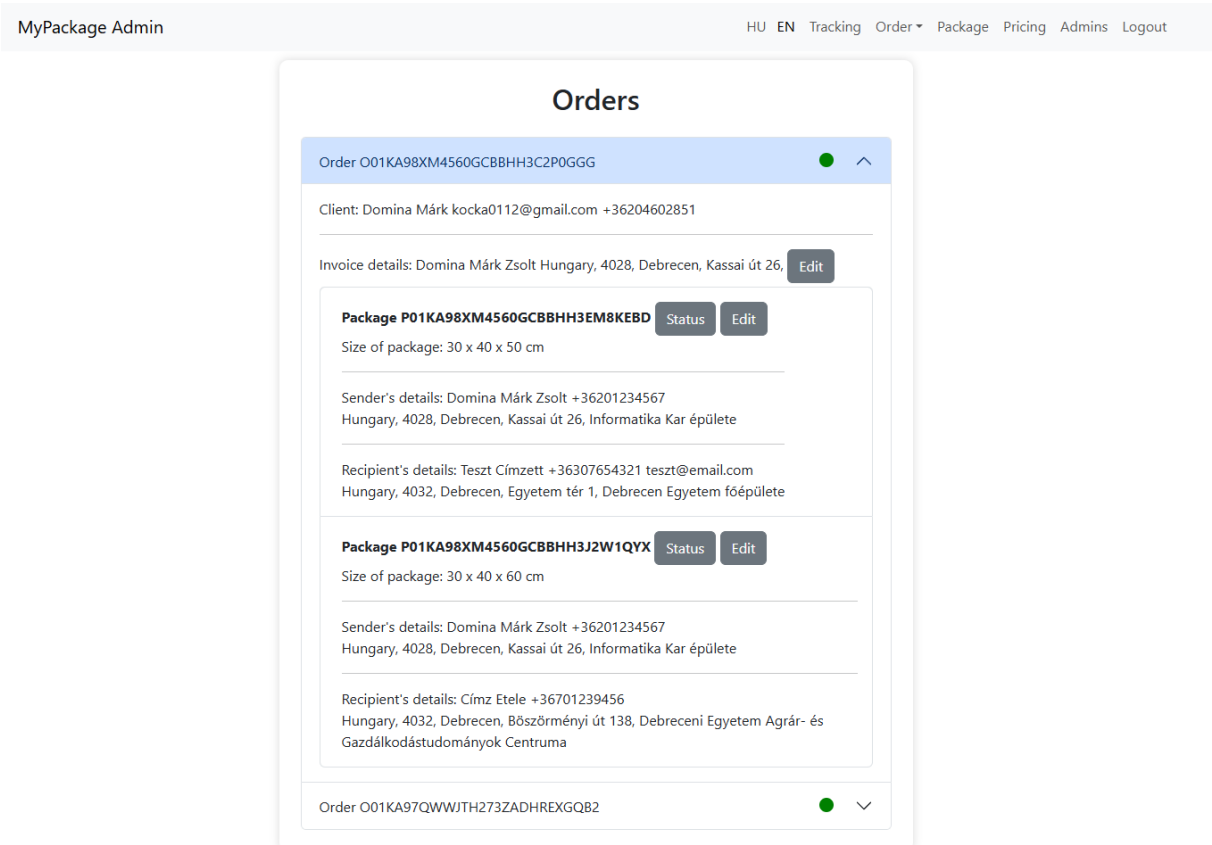
Amennyiben a felhasználó befejezte a munkáját, akkor a nevére való kattintással lenyíló menüből ki tud jelentkezni.

7.2.2 Adminisztrációs felület

Az adminisztrációs felület megnyitásakor egy egyszerű bejelentkező felület fogadja az adminisztrátorokat. Itt nincsen lehetőség adminisztrációs fiókok regisztrációjára. Az alapértelmezett adminisztrátor felhasználóval (felhasználónév: admin, jelszó: admin) lehet először csak bejelentkezni. Az adminisztrátorok kezelőfelületeihez csak bejelentkezés után lehet hozzáférni, amelyek között a navigációs sáv segítségével lehet lépkedni.

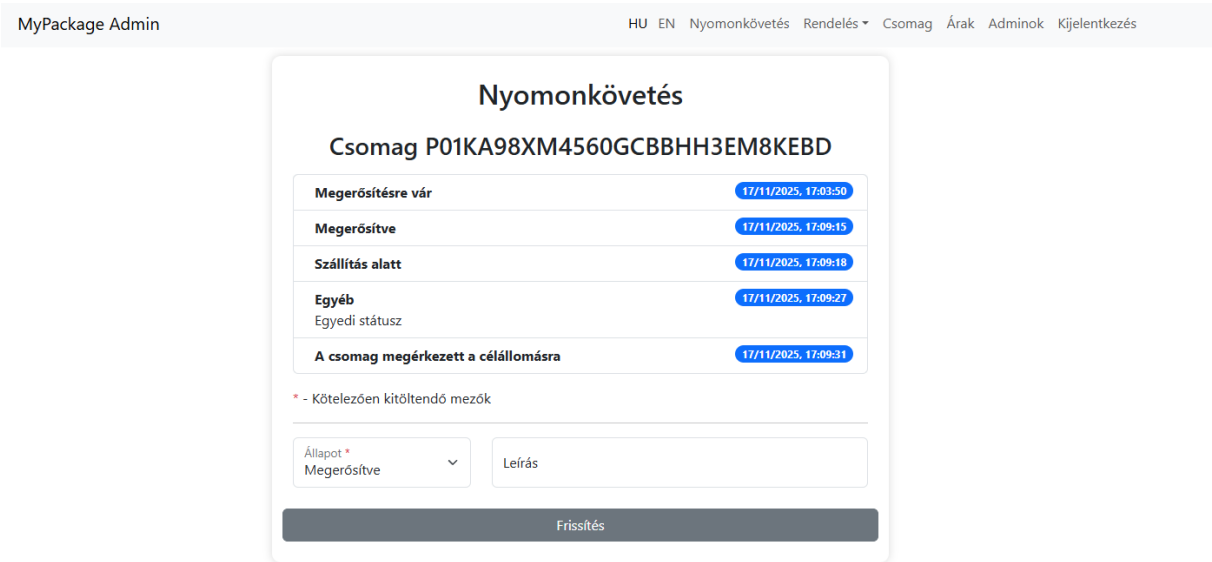
Először az összes rendelést kilistázó felületet lehet érdemes megnyitni. Ehhez a rendelés nevű lenyíló menüből a rendeléseket kell megnyitni. Ez a felület is a felhasználókéhoz hasonlóan egy harmonikákból álló listából áll, de harmonikákon belül található szerkesztés és állapot gombokkal lehetőség van ezeknek a rendelések, csomagok adatainak módosítására. (34. ábra)

A rendelések azonosítói mellett található zöld/piros lámpa jelzi, hogy rendelés éppen aktív-e.



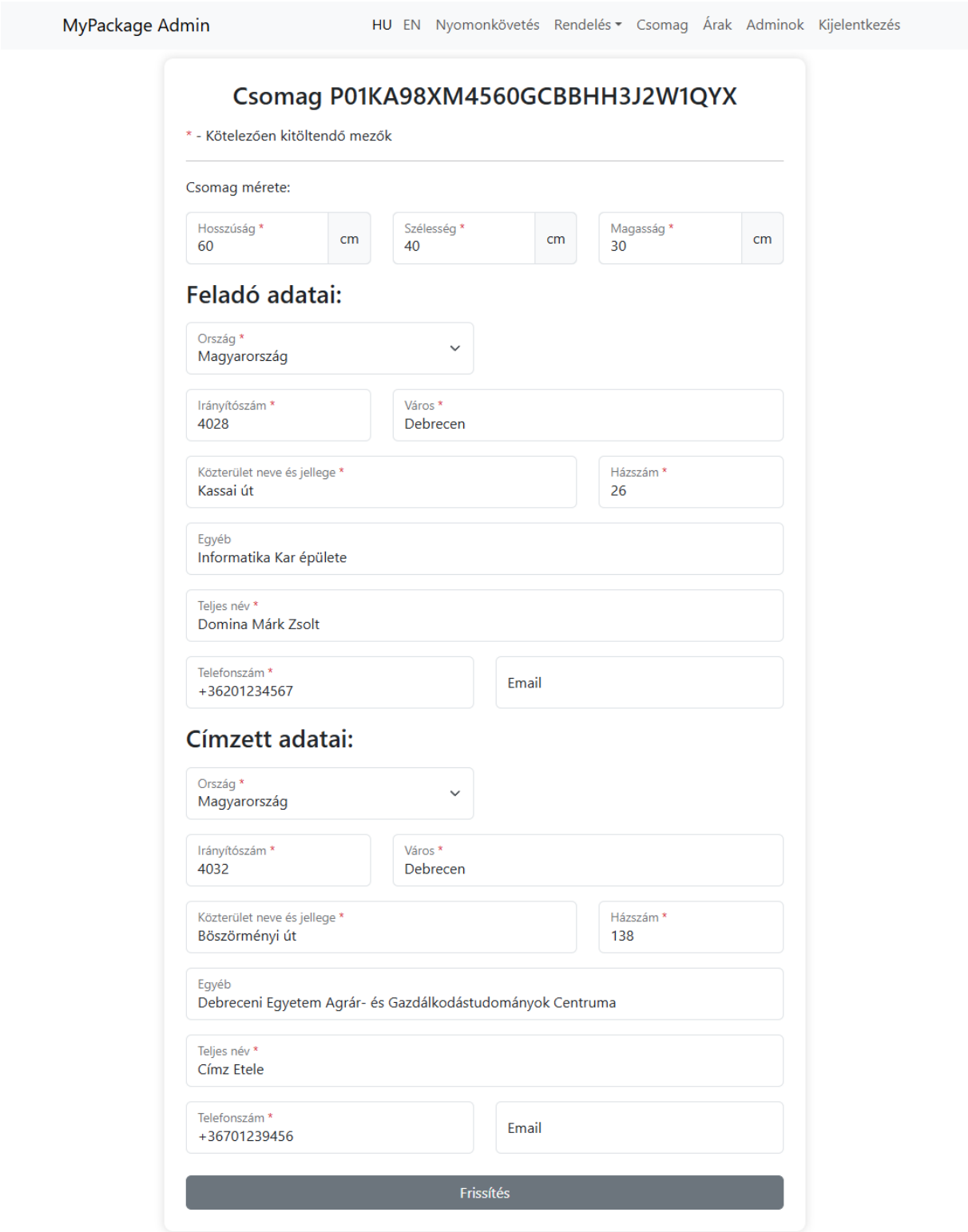
34. ábra: A rendszerben található összes csomagot kilistázó felület

Az állapot gombra kattintva megjelenik a csomaghoz tartozó nyomonkövetés kezelőfelület. (33. ábra) Ezt az oldalt oldalt nem csak a rendelésekből lehet elérni, navigációs sávból is. Ezután a csomagazonosító megadásával lehet megnyitni a kezelőfelületet. A kezelőfelületen megtekinthetőek a már rögzített állapotok, de itt tudunk új státuszokat is rögzíteni. Új állapot létrehozásánál az előre megadott állapotok közül lehet választani, illetve opcionálisan leírást is meg lehet adni.



33. ábra: Csomag állapot kezelőfelület

Ha a rendelés kilistázásnál valamelyik szerkesztés gombra kattintottunk, akkor az ahhoz tartozó csomag vagy rendelés kezelőfelületén lyukadunk ki. (34. ábra) A csomag és rendelés kezelőfelület hasonlítanak egymáshoz: megjelenő űrlapba be vannak töltve az aktuális értékek, és ezek megváltoztatása után a frissítés gombbal lehet elmenteni az új adatokat. Ezeket a kezelőfelületeket a navigációs sávból is el lehet érni, ahol szintén a megfelelő azonosítókat kell megadni az űrlapok megjelenítéséhez.



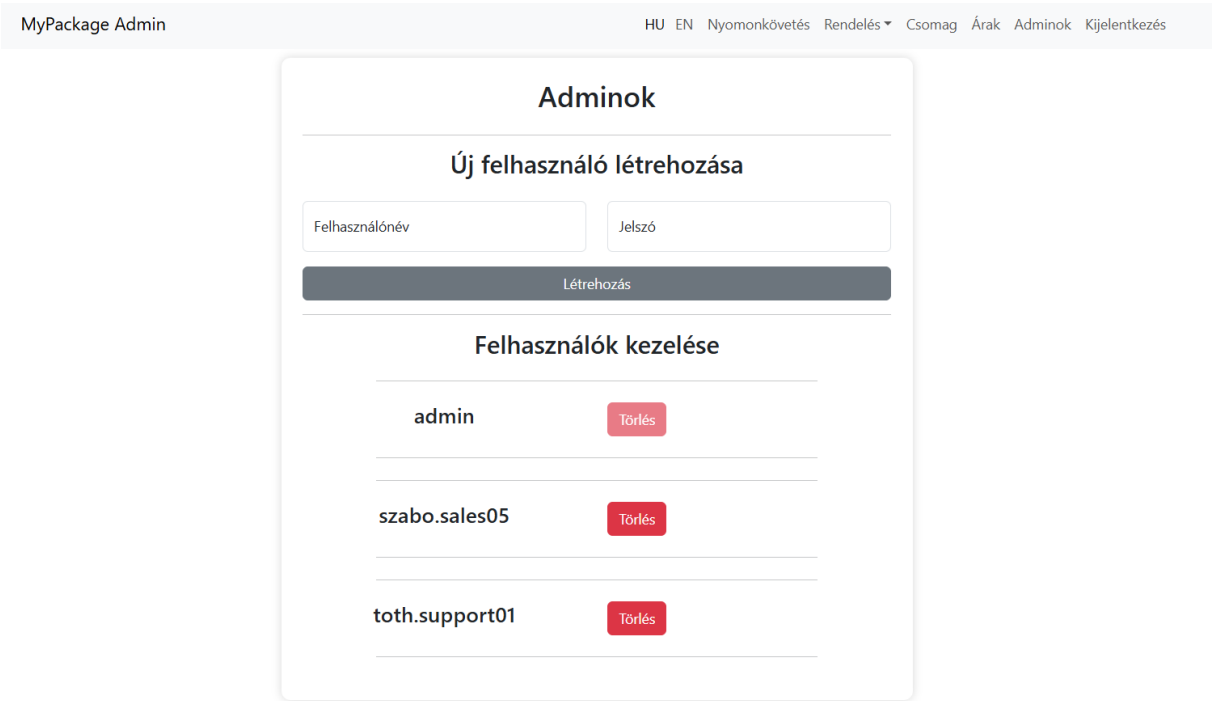
34. ábra: Csomag adatait módosító kezelőfelület

Új adminisztrációs fiókok megadásához és törléséhez a navigációs sávból elérhető adminok felületet kell megnyitni. (35. ábra) Ezt az oldalt csak az alapértelmezett felhasználó tudja elérni.

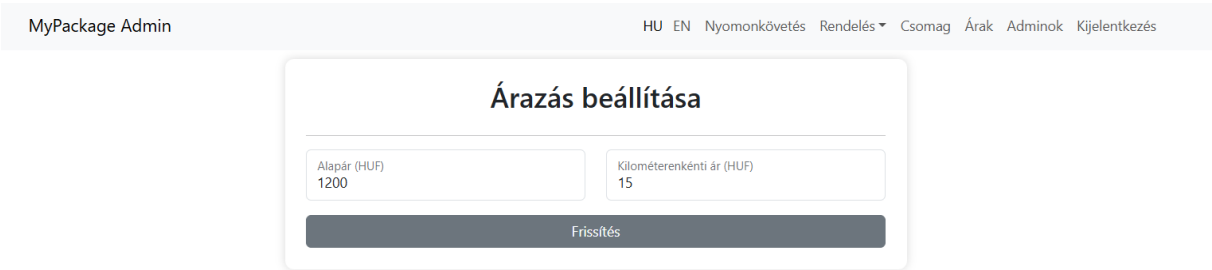
Új fiók létrehozásához egy felhasználónevet és jelszót kell megadni, majd a létrehozás gombra nyomni. Felhasználói fiók törléséhez a törölni kívánt felhasználó neve melletti piros törlés gombra kell kattintani.

Az adminisztrátorok a csomagküldés számításához használt árakat is tudják változtatni. Ehhez az Árak felületet kell megnyitni. Az oldalt megnyitva az Alapár és Kilóméterenkénti ár beviteli mezőbe betöltődnek a jelenlegi értékek, az értékek megváltoztatásai után a frissítés gombbal lehet menteni az új árakat. (36. ábra)

Amennyiben az adminisztrátor befejezte a munkáját, a navigációs sávban található kijelentkezés gombbal meg tudja szakítani a munkamenetét.



35. ábra: Adminisztrátor felhasználók kezelőfelülete



36. ábra: Árazás kezelőfelülete

8 Összefoglalás

A szakdolgozatom elején két nagy célt tűztem ki. Az egyik cél az volt, hogy különböző csomagküldő vállalatok weboldalait megvizsgálata után következtetéseket vonjak le arról, hogy milyen funkciókkal kell rendelkeznie ilyen webalkalmazásoknak, amit sikeresen el is végeztem, és szakdolgozatom első felében kifejtettem. A legtöbb megállapítást az oldalak kinézetéről tudtam tenni, tüzetesen megvizsgáltam az oldalak felépítését, tartalmát. Sikerült rámutatni különböző vállalatok oldalainak közötti különbségekre, például az űrlapok kinézetére, működésére, de közös tulajdonságokat is találtam, például az oldalak többnyelvűsége, csomag nyomonkövetési szolgáltatások, árkalkulátorok. Olyan funkcióra is leltem, amelyet a vizsgált oldalak közül csak egy valósított meg, az egyszerre több csomag feladásának a lehetőségét, amit az alkalmazásomban is megvalósítottam.

A dolgozatom másik célja az volt, hogy a kutatási eredményeim alapján egy olyan webalkalmazást készítsek, amellyel egy fiktív csomagszállító vállalat ügyfelei és ügyfélszolgálata között egy eseti csomagrendelésre alkalmas kommunikációs felület jöjjön létre. Az általam leprogramozott rendszerben a felhasználók képesek csomagrendelések leadására, és a rendelések információit az adminisztrátorok fel tudják dolgozni, és visszajelzést is tudnak adni a nyomonkövetésen keresztül.

Az alkalmazás úgy lett kialakítva, hogy a későbbiekben lehessen új funkciókat is hozzáadni, de akár mikroszolgáltatások bevezetésére alkalmas, emellett olyan alapvető funkciókra is képes, mint az egyedi konfiguráció kezelése, naplózás, metrikák begyűjtése. A fejlesztés során sok új technológiával és módszerrel is megismerkedtem, és modern fejlesztői eszközök használatával zajlott. A weboldalak reszponzív dizájnja véleményem szerint nagyon jól használható lett, és az oldalak kinézetébe sikerült beleépítenem a kutatásom során felmerült funkciókat is.

Van két témakör, amikről úgy gondolom, hogy nem foglalkoztam velük eleget, az egyik a mobilos felületek elkészítése. Habár a megvalósított PWA is tökéletesen használható, egy natív mobilalkalmazás programozása lehet jobb lett volna teljesítmény és UI/UX szempontból is, de ezt igazán úgy lehetne lemérni, ha ugyanazt a PWA és natív alkalmazást össze tudnám hasonlítani. A Prometheus szerver működését kipróbáltam a fejlesztés során, ami sikeresen látja a begyűjtött metrikákat, de érdemi kimutatásokat a rendszer teljesítményéről nem sikerült készítenem.

Úgy gondolom, hogy az elkészült alkalmazásommal teljesítettem a célkitűzésemet, de a kész rendszer nem alkalmas arra, hogy ebben a formájában egy valós csomagküldő szolgálat

informatikai rendszere legyen. Olyan kritikus szolgáltatások nélkül, mint a számlázás, leltárnyilvántartás és fuvarszervezés nem tudna egy ilyen vállalat üzemelni, és emellett számos kényelmi funkcióra is igény merülhetne fel, például egy HR rendszerre vagy kiszállító járművek nyomonkövetésére. A szakdolgozat elkezdésekor is tisztában voltam vele, hogy nem tűzhetem ki célnak egy teljes vállalatirányítási rendszer fejlesztését.

Összeségében elégedett vagyok az elvégzett munkámmal, és remélhetőleg az olvasó is hasznosnak fogja találni a megfigyeléseimet, vagy az elkészült alkalmazásomat. A szakdolgozatom elkészítése során számos új szakmai és csomagszállítással kapcsolatos tudásra tettem szert, amit a jövőbeli projektjeim során is kamatoztathatok majd.

9 Köszönetnyilvánítás

Szeretném megköszönni témavezetőmnek, Dr. Major Sándor Rolandnak a szakdolgozatom elkészítéséhez nyújtott segítségét és rugalmasságát.

10 Felhasznált könyvtárak

A backendhez felhasznált nem sztenderd Go csomagok neveinek listája: (go.mod fájlban is megtalálhatóak)

github.com/alexedwards/argon2id
github.com/go-kit/kit
github.com/go-passwd/validator
github.com/golang-jwt/jwt/v4
github.com/google/uuid
github.com/jftuga/geodist
github.com/nyaruka/phonenumbers
github.com/oklog/ulid/v2
github.com/prometheus/client_golang
github.com/rs/cors
golang.org/x/sync
golang.org/x/time
gopkg.in/gomail.v2
gopkg.in/yaml.v3
gorm.io/driver/postgres
gorm.io/gorm

A frontendhez felhasznált, a SvelteKit függőségeit nem tartalmazó npm csomagok listája:

@inlang/paraglide-js
validator

11 Irodalomjegyzék

[1] KSH, „KSH STADAT 12.1.1.16. Az internethasználat gyakoriságának megoszlása," 2024. [Online]. Available: https://www.ksh.hu/stadat_files/ikt/hu/ikt0018.html. [Hozzáférés dátuma: 15 10 2025].

[2] KSH, „KSH STADAT 12.1.1.17. Az internetes vásárlások aránya az utolsó vásárlás időpontja szerint," 2024. [Online]. Available: https://www.ksh.hu/stadat_files/ikt/hu/ikt0019.html. [Hozzáférés dátuma: 15 10 2025].

[3] KSH, „KSH STADAT 24.1.1.15. Postai és futárpostai küldemények," 2024. [Online]. Available: https://www.ksh.hu/stadat_files/sza/hu/sza0015.html. [Hozzáférés dátuma: 15 10 2025].

[4] GLS, „GLS cégcsoport bemutatása," [Online]. Available: https://gls-group.com/HU/hu/vallalat/gls-cegcsoport/. [Hozzáférés dátuma: 10 11 2025].

[5] ExpressOne, „ExpressOne Rólunk," [Online]. Available: https://expressone.hu/rolunk. [Hozzáférés dátuma: 10 11 2025].

[6] UPS, „Home | About UPS," [Online]. Available: https://about.ups.com/gb/en/home.html. [Hozzáférés dátuma: 10 11 2025].

[7] DHL, „DHL Rólunk," [Online]. Available: https://www.dhl.com/hu-hu/home/rolunk.html. [Hozzáférés dátuma: 10 11 2025].

[8] Foxpost, „Foxpostról," [Online]. Available: https://foxpost.hu/foxpostrol. [Hozzáférés dátuma: 10 11 2025].

[9] D. D. D. K. S. J. Alex Biryukov, „Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," RFC Editor, 9 2021. [Online]. Available: https://datatracker.ietf.org/doc/rfc9106/. [Hozzáférés dátuma: 15 11 2025].

[10] Go, „The Go Programming Language Specification Language version go1.25," 12 9 2025. [Online]. Available: https://go.dev/ref/spec. [Hozzáférés dátuma: 17 11 2025].

[11] Gokit, „Frequently asked questions," Go kit, [Online]. Available: https://gokit.io/faq. [Hozzáférés dátuma: 18 11 2025].

[12] J. B. N. S. Michael B. Jones, „JSON Web Token (JWT)," RFC Editor, 5 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519. [Hozzáférés dátuma: 15 11 2025].

[13] Inlang, „Paraglide JS," [Online]. Available: https://inlang.com/m/gerre34r/library-inlang-paraglideJs. [Hozzáférés dátuma: 13 11 2025].

[14] Inlang, „Paraglide JS Basics," [Online]. Available: https://inlang.com/m/gerre34r/library-inlang-paraglideJs/basics. [Hozzáférés dátuma: 13 11 2025].

[15] DB-Engines, „DB-Engines Ranking," 11 2025. [Online]. Available: https://db-engines.com/en/ranking. [Hozzáférés dátuma: 17 11 2025].

[16] PostgreSQL, „About," [Online]. Available: https://www.postgresql.org/about/. [Hozzáférés dátuma: 17 11 2025].

[17] MDN, „Progressive web apps," MDN, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps. [Hozzáférés dátuma: 17 11 2025].

[18] MDN, „Making PWAs installable," MDN, [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps/Guides/Making_PWAs_installable. [Hozzáférés dátuma: 17 11 2025].

[19] Prometheus, „Prometheus Overview," [Online]. Available: https://prometheus.io/docs/introduction/overview/. [Hozzáférés dátuma: 13 11 2020].

[20] Svelte, „Overview," Svelte, [Online]. Available: https://svelte.dev/docs/svelte/overview. [Hozzáférés dátuma: 18 11 205].

[21] Svelte, „.svelte files," Svelte, [Online]. Available: https://svelte.dev/docs/svelte/svelte-files. [Hozzáférés dátuma: 18 11 2025].

[22] Svelte, „What are runes?," Svelte, [Online]. Available: https://svelte.dev/docs/svelte/what-are-runes. [Hozzáférés dátuma: 18 11 2025].

[23] SvelteKit, „Routing," Svelte, [Online]. Available: https://svelte.dev/docs/kit/routing. [Hozzáférés dátuma: 18 11 2025].

[24] SvelteKit, „Hooks," Svelte, [Online]. Available: https://svelte.dev/docs/kit/hooks. [Hozzáférés dátuma: 18 11 2025].

[25] SvelteKit, „Adapters," Svelte, [Online]. Available: https://svelte.dev/docs/kit/adapters. [Hozzáférés dátuma: 18 11 2025].

[26] YAML, „YAML Ain't Markup Language (YAML™) version 1.2," [Online]. Available: https://yaml.org/spec/1.2.2. [Hozzáférés dátuma: 13 11 2025].

12 Függelék

12.1 HTTP API Végpontok listája

12.1.1 Felhasználó API

HTTP kérés típusa	Útvonal	Cél	Auth
POST	/login	Felhasználó bejelentkezés	Nem
POST	/register	Felhasználó regisztráció	Nem
POST	/logout	Felhasználó kijelentkezés	Igen
GET	/refresh	Felhasználói munkamenet frissítése	Igen
POST	/price	Ár kalkulálása	Nem
POST	/order/create	Új rendelés létrehozása	Igen
GET	/get	Egy adott adat/rendelés lekérése	Igen
POST	/verify	Fiók megerősítése	Nem
GET	/track	Csomagkövetési státusz lekérése	Nem
GET	/getall	Összes felhasználói rendelés lekérése	Igen

12.1.2 Admin API

HTTP kérés típusa	Útvonal	Cél	Auth
GET	/admin/get	Admin információ lekérése	Igen
POST	/admin/login	Admin bejelentkezés	Nem
POST	/admin/logout	Admin kijelentkezés	Igen
GET	/admin/refresh	Admin munkamenet frissítése	Igen
GET	/admin/getall	Összes felhasználó lekérése	Igen
DELETE	/admin/user	Felhasználó törlése	Igen
PUT	/admin/user	Felhasználó létrehozása vagy frissítése	Igen
GET	/admin/order	Egy adott rendelés lekérése	Igen

HTTP kérés típusa	Útvonal	Cél	Auth
GET	/admin/orders	Összes rendelés lekérése	Igen
DELETE	/admin/order	Rendelés törlése	Igen
POST	/admin/order	Rendelés frissítése	Igen
GET	/admin/package	Csomagadatok lekérése	Igen
DELETE	/admin/package	Csomag törlése	Igen
POST	/admin/package	Csomag frissítése	Igen
POST	/admin/status	Státusz hozzáadása	Igen
GET	/admin/pricing	Árkonfiguráció lekérése	Igen
POST	/admin/pricing	Árazás beállítása vagy módosítása	Igen
GET	/metrics	Prometheus metrikák	Nem

12.2 Alkalmazásból elérhető weboldalak listája

Útvonal	Oldal tartalma (rövid leírás)
/	Kezdőlap
/admin	Admin bejelentkezés
/admin/orders	Admin – Rendelések listája
/admin/order	Admin – Rendelés részletei
/admin/package	Admin – Csomag módosítása
/admin/pricing	Admin – Árazás kezelése
/admin/status	Admin – Csomagállapot kezelése
/admin/users	Admin – Admin Felhasználók kezelése
/app	Felhasználói alkalmazás bejelentkezés
/app/faq	Felhasználói GYIK
/app/orders	Felhasználó rendelései
/app/order	Rendelés leadása

Útvonal	Oldal tartalma (rövid leírás)
/app/pricing	Árkalkulátor
/app/tracking	Csomag státuszának követése